TM Forum Introductory Guide

Agentic-AI Service-Management Framework Aligned with TM Forum Intent Ontology (TIO)

IG1421A

Maturity Level: Beta	Team Approved Date: 10-Sep-2025
Release Status: Pre-production	Approval Status: Member Evaluated
Version 1.1.0	IPR Mode: RAND

©TM Forum 2025. All Rights Reserved.

Notice

Copyright © TM Forum 2025. All Rights Reserved.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to TM FORUM, except as needed for the purpose of developing any document or deliverable produced by a TM FORUM Collaboration Project Team (in which case the rules applicable to copyrights, as set forth in the TM FORUM IPR Policy, must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by TM FORUM or its successors or assigns.

This document and the information contained herein is provided on an "AS IS" basis and TM FORUM DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Direct inquiries to the TM Forum office:

100 Enterprise Drive, Suite 301 #1649
Rockaway, NJ 07866, USA
Tel No. +1 862 227 1648
TM Forum Web Page: www.tmforum.org

Table of Contents

Notice 2
Table of Contents 3
List of Tables 6
List of Figures 8
Executive Summary 9
Introduction 10
1. Scope and Objectives 12
1.1. Scope 12
2. AI Agent MAS Taxonomy of AN Service Management 14
2.1. AI Agent Functional Role 14
2.2 Decision-Making Autonomy 16
2.3. AI Agent Interaction Model 18
2.4. AI Agent Deployment Scope 19
3. AI Agent MAS Intent-Based Interaction Model 22
3.1. Autonomous AI Agents 22
3.2. Collaborative AI Agents 23
3.3. Hierarchical AI Agents 23
3.4. Centralized Coordination Agents 24
3.5. Distributed Coordination Agents 24
3.6. Market-Based Coordination AI Agents 25
3.7. Contract-Based Coordination AI Agents 26
3.8. Coalition-Based Coordination AI Agents 26
3.9. AI Agent MAS Simulation & Inference as a Coordination Mechanism 28
3.10. Self-Healing in Intent-Based AI Agent MAS Service Management 31
4. Cross-Domain Coordination & AD Roles 33
4.1. Multi-Agent System (MAS) Coordination 35
5. Intent Lifecycle & MAS Coordination 38
5.1. TIO Lifecycle Concepts 38
5.2. Phase 1 - Discovery 39

© TM Forum 2025. All Rights Reserved.

Phase 2 — Negotiation 40
5.3. Phase 3 — Fulfillment 41
5.4. Feedback Flows 41
5.5. Ontology Lifecycle 42
5.6. TIO Lifecycle Events 43
6. AI Agent MAS and Intent Management Function (IMF) 44
6.1. AI Agents and Network & Resource Management 44
6.2. Intent-Based MAS Framework across AD Layers 44
6.3. Knowledge-Graph Representation of Intent State 45
6.4. AI-Agent Roles in Graph Updates (A-Box) 46
6.5. Semantic intent vs. operational execution 46
6.6. Key Roles in the Intent Pipeline 46
7. Intent-Driven Autonomous Domains using TIO 48
7.1. Ontology representation components 48
8. Intent Specification & Template Mapping 50
8.1. Parameter Representation 50
8.2. Declarative vs Operational Layers 50
8.3. API Integration (TMF Open APIs) 51
8.4. Validation & SHACL Hooks 52
9. IMF and AD – Agentic-AI Protégé Ontology – TIO 57
Turtle File for MAS Agentic-AI Protégé Ontology 62
10. Summary 66
10.1. Key MAS Parts 67
10.2. Conclusion & Outlook 67
11. Appendix A: MAS Agentic-AI Intent-Based AN Operations – Formulation and Analytics 69
11.1. Agentic-AI MAS AN 69
11.2. Agentic-AI Enabled Intent Management 70
11.3. AD Hierarchy Model 71
Service ont:UtilityFunction 73
Cross-Domain Coordination and Negotiation Model 75
Intent Management Function (IMF) Model 76

© TM Forum 2025. All Rights Reserved.

12. Appendix B: Simulation vs. Emulation vs. Digital Twin – Tradeoffs in Resource-Level Coordination 83
12.1. Hybrid Telemetry / Partial-Twin Flow 84
12.2. Best Practices 86
13. Appendix C: Integration of AI Agent MAS Framework 88
14. Appendix D: Carrier-Influenced BGP Path Selection 90
15. Appendix E: T-Box and A-Box 91
T-Box (Terminological Box) 91
A-Box (Assertional Box) 91
16. Appendix F: Standards-Aligned Intent Execution Template 93
Intent Model RFC 9315 and RFC9316 93
TMF Intent Model Artifacts and Alignment 96
17. Terms & Abbreviations Used within this Document 97
17.1. Terminology 97
17.2. Abbreviations & Acronyms 97
18. References 98
19. Administrative Appendix 103
19.1. Document History 103
19.2. Acknowledgments 103

© TM Forum 2025. All Rights Reserved.

List of Tables

Table 1: Agentic-AI MAS Key Capabilities 12
Table 2: AI Agent Functional Role 15
Table 3: Decision Making Autonomy 18
Table 4: Interaction Model 19
Table 5: Deployment Scop 21
Table 6: MAS Coordination Type 27
Table 7: Sample MAS Coordination Type 27
Table 8: AI Agent MAS Interaction Use Case and Applications Domain 28
Table 9: Comparison of Inference and Simulation 31
Table 10: Cross Domain Coordination using IMF 33
Table 11: Primary Intent Type and IMF view 33
Table 12: Intent Lifecycle 38
Table 13: Feedback Flows 42
Table 14: Ontology Lifecycle turtle syntax 42
Table 15: Autonomy Classes 44
Table 16: Autonomy Layer Domain 44
Table 17: TIO Use Cases 45
Table 18: AI Agent Role 46
Table 19: TIO Prefixes 48
Table 20: Ontology Components 48
Table 21: Autonomous Domain Responsibility 49
Table 22: Intent flow across ADs 49
Table 23: Parameter Representation 50
Table 24: Declarative vs. Operation 50
Table 25: A Template Example 51
Table 26: Template 51
Table 27: API Integration 51
Table 28: Linkage Pattern 52
Table 29: Intent Compliance Shape 52
Table 30: API Boundary Shape 53
Table 31: MAS Class Hierarchy Turtle Syntax 55
Table 32: Ontology Prefix and Namespace 58

© TM Forum 2025. All Rights Reserved.

Table 33: Key object properties 59
Table 34: TMF Alignment 61
Table 35: Turtle syntax of the complete Ontology 62
Table 36: Summary of the Main Contribution 66
Table 37: Opportunities and Standards Gaps 68
Table 38: CFS and RFS Function Transformation 74
Table 39: Diagram Layer Outline Description 85
Table 40: AI Agent Module Architecture 86
Table 41: Simulation, Emulation and Digital Twin Comparison 87
Table 42: Formal Representation Snippet (OWL/Turtle-Aligned) 88
Table 43: Summary of T-Box and A-Box 91
Table 44: Intent Model 93
Table 45: Intent Field Description 94
Table 46: Intent Functional Block 94
Table 47: Observability Points 95
Table 48: Performance Optimization Intent for RAN 95
Table 49: TMF Intent Artifact 96

© TM Forum 2025. All Rights Reserved.

List of Figures

Figure 1: Layered Intent AI-Agent Based Service Management Architecture.....11

Figure 2: AI Agent MAS Interaction Model across product, service and resource domains 35

Figure 3: Lifecycle Phases.....39

Figure 4: Intent-to-Action Pipeline 47

Figure 5: Semantic Execution of Network Intent 47

Figure 6: TIO MAS Protege Class Hierarchy 54

Figure 7: TIO Protégé IMF for AD – Agentic-AI MAS Ontology.....57

Figure 8: Three concepts intersect and complement one another in resource-level orchestration. 83

Figure 9: Hybrid Telemetry and Partial Twin Flow.....85

© TM Forum 2025. All Rights Reserved.

Executive Summary

Autonomous Networks (ANs) are shifting from static, rule-based workflows to intent-driven operations that require machine-readable goals, adaptive enforcement, and seamless cross-domain coordination.
This guide introduces an Agentic-AI Service-Management Framework—fully aligned with the TM Forum Intent Ontology (TIO)—that uses a Multi-Agent System (MAS) to *express*, *translate*, *negotiate*, and *operationalize* intent across the product, service, and resource layers.
The framework equips each Autonomous Domain (AD) with closed-loop automation for policy adaptation, fault recovery, and real-time assurance, enabling holistic governance of distributed network domains.

Intent values captured during product- and service-ordering flow through a well-defined life-cycle: product-level intents tied to Customer-Facing Services (CFSs) are decomposed into operational intents for Resource-Facing Services (RFSs), ensuring fulfilment and assurance remain traceable from the customer goal down to individual network functions. This vertical propagation of goals delivers end-to-end compliance across every operational layer.

End-to-end service management is orchestrated by the Intent-Management Function (IMF) defined in TMF IG1253 [1]. The IMF distributes KPI-based delivery expectations to subordinate domains so that all downstream CFS and RFS intents stay aligned with the originating product requirement.
The framework relies on TM Forum Open APIs [2 | 4 | 6 | 7] to formalize AI-agent interactions, hierarchical intent propagation, and self-optimizing behavior. It addresses four pillars:

1. AI-Agent MAS Taxonomy & Ontology – functional roles, semantic model, autonomy tiers, interaction modes, deployment scope [9].
2. Intent-based AI Coordination – game-theoretic negotiation and Markov-Decision-Process (MDP) reasoning for dynamic conflict resolution [32].
3. Closed-loop Self-optimization – adaptive control across product, service, and resource layers.
4. Standardized API Integration – TMF620 (Product Catalog), TMF633 (Service Catalog), TMF915 (AI Management), TMF921 (Intent Management) for scalable, vendor-neutral automation.

By merging semantic intent, agentic orchestration, and TMF Open APIs into one cohesive stack, operators can realize intelligent, interoperable, and autonomous service operations that adapt in near real-time to customer demands and network conditions.

© TM Forum 2025. All Rights Reserved.

Introduction

Autonomous Networks (ANs) are moving beyond static, rule-driven workflows toward intent-driven service management—a paradigm in which network behavior is guided by high-level goals ("intents") rather than low-level configuration scripts. Realizing this vision at scale demands an Agentic-AI approach: a Multi-Agent System (MAS**)** of specialized AI-agents that can interpret, negotiate, and enforce intent across heterogeneous technology domains.

This document presents a unified Agentic-AI Service-Management Framework aligned with the TM Forum Intent Ontology (TIO) and TM Forum's Intent-Management Function (IMF) blueprint (IG1253) [1], TMF1251 [2], and Ericsson Technology Review "Evolving service management toward intent-driven autonomous networks" [3] (Ericsson Review, "Evolving Service Management Toward Intent-Driven Autonomous Networks", Nov 2024) captures both resource-level and service-level autonomous operations.. The framework contributes:

- TIO-aligned ontology and life-cycle model—defining classes for AI-agents, lifecycle phases, Autonomous Domains (ADs) and intent artefacts, plus properties for delegation, negotiation, and metric linkage.
- MAS taxonomy—classifying agents by resource-management focus (optimization, assurance, security, recovery), decision style (rule-based, goal-driven, learning, hybrid), interaction pattern (autonomous, collaborative, hierarchical) and deployment scope (local, global, cross-domain).
- Closed-loop execution architecture—each AD embeds an IMF that supervises local agent swarms and exchanges intent with peer IMFs, enabling vertical decomposition (Product → Service → Resource) and horizontal coordination.
- API backbone—intent semantics flow through TM Forum Open APIs (TMF620 Product-Catalog [4], TMF633 Service-Catalog [5], TMF921 Intent-Management [8], TMF915 AI-Management), making every agent action traceable and interoperable.

Intent flow and agent roles

1. Capture – A customer's goal is recorded as a **ig:ProductIntent** during product ordering and linked to CFS entries in TMF620 [4] and TMF633 [5].
2. Decompose – The Product-level IMF, acting as *intent owner*, uses ig:hasSubIntent to break that goal into ig:ServiceIntent objects, distributing KPI constraints to Service-level agents.
3. Translate & Negotiate – ig:IntentTranslationAgent and ig:IntentNegotiationAgent assess feasibility and, if necessary, negotiate trade-offs with peer IMFs via **ig:interADNegotiation** links.
4. Enforce – ig:IntentHandlerAgent in each Resource-level AD instantiates **ig:ResourceIntent** objects and calls southbound controllers through TMF641 and TMF 642, while ig:FaultRecoveryAgent and

© TM Forum 2025. All Rights Reserved.

ig:IntentMonitoringAgent close the MADE (Monitor–Analyze–Decide–Execute) loop with real-time metrics (met:RuntimeMetric).

5. Assure & Optimize – Utility functions (mf:UtilityFunction) guide self-optimization; monitoring agents publish compliance reports, allowing IMFs to refresh delivery expectations without human intervention.

Agentic-AI is essential for the following:

- Scalability – Hundreds of micro-intents emerge from dynamic slicing, MEC workloads and customer QoS tiers; a single "super-agent" cannot track them.
- Heterogeneity – Access, transport and core domains run different controllers; specialized agents abstract those differences while IMFs provide a common negotiation layer.
- Adaptivity – Learning-based agents continuously refine policies to maintain KPIs amid traffic surges, failures or topology changes.
- Interoperability – By grounding every interaction in TIO semantics and TM Forum APIs, multiple vendors' agents can cooperate without custom glue code.

Throughout this guide, figures and Turtle examples (see Figure **1** and Appendixes) show how intents propagate, how AI-agents coordinate, and how delivery expectations are enforced end-to-end—demonstrating a practical path from standards-defined intent models to fully autonomous, Agentic-AI operations.

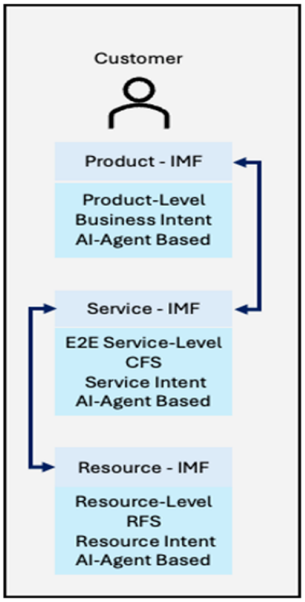

Figure 1: Layered Intent AI-Agent Based Service Management Architecture

© TM Forum 2025. All Rights Reserved.

1. Scope and Objectives

This specification presents an Agentic-AI Service-Management Framework for ANs. The framework combines a MAS of specialized AI-agents with the IMF model, providing the semantic, ontological, and life-cycle bindings needed for interoperable deployments over TM Forum Open APIs.

1.1. Scope

- Intent formalization – express customer and operator goals as machine-readable, ontology-grounded objects (goals, constraints, desired outcomes).
- Agentic-AI role model – define AI-agent roles that cover the entire intent life-cycle: discovery, translation, negotiation, monitoring, assurance, and optimization.
- Decision & coordination methods – apply game-theoretic negotiation, Markov-Decision-Processes, and reinforcement learning so agents can resolve conflicts and optimize policies inside and across Autonomous Domains.
- API integration – bind every agent action to TM Forum Open APIs (TMF620 Product-Catalog [4], TMF633 Service-Catalog [5], TMF915 AI-Management [7], TMF921 Intent-Management [8], etc.) to guarantee programmatic interoperability and automation.

Key capabilities delivered

Table 1: Agentic-AI MAS Key Capabilities

Capability	How the Agentic-AI MAS achieves it
Cross-domain coordination	ig:IntentNegotiationAgent instances exchange proposals via ig:interADNegotiation, aligning product, service, and resource intents end-to-end.
Adaptive policy enforcement	ig:IntentHandlerAgent applies closed-loop OODA (Observe-Orient-Decide-Act) logic driven by real-time metrics.
Self-healing & assurance	ig:FaultRecoveryAgent and ig:IntentMonitoringAgent detect deviations and trigger automated remediation.
Security controls	Security-focused agents enforce slice isolation and adaptive ACLs using TMF915 policy hooks.
Semantic interoperability	A TIO-aligned ontology ensures every intent, metric and API call is machine-understandable across vendor boundaries.
Scalable orchestration	Autonomous Domains (ig:ProductLevelAD, ig:ServiceLevelAD, ig:ResourceLevelAD) each host an IMF that supervises local agent swarms, enabling horizontal scaling without a single point of control.

© TM Forum 2025. All Rights Reserved.

Objective: enable operators to deploy agentic, self-optimizing, intent-aware networks where AI-agents negotiate goals, invoke TMF APIs, and keep services compliant in near real-time with minimal human intervention.

© TM Forum 2025. All Rights Reserved.

2. AI Agent MAS Taxonomy of AN Service Management

The taxonomy organizes AI agents according to the roles they perform, their decision-making autonomy, how they interact, and the scope of their deployment. This structure helps CSP reason about which agent capabilities are required at each layer of an autonomous, intent-driven environment.

Agents collaborate to support optimization, performance and service quality management, security enforcement, and fault recovery across dynamic networks and services—all aligned to user and business objectives.

The taxonomy provides a comprehensive framework for understanding the roles, capabilities, and interactions of AI agents in intent-based ANs. These agents work together as part of a MAS to support system optimization, performance management, service quality management, security enforcement, and fault recovery, all driven by the dynamic needs of the system and its users.

The taxonomy of AI agents classifies and structure the agent based on their

- Functional roles
- Decision-making abilities
- Interaction models, and
- Deployment scope.

This follows the principles of taxonomy: categorization, classification, and structuring knowledge in a clear, hierarchical manner.

2.1. AI Agent Functional Role

Perception & Awareness**.** Agents gather raw telemetry, logs, and environmental signals, then enrich and structure that data into usable information for downstream decision logic—an essential preprocessing step in adaptive automation as summarized in Table 2.

Intent is not limited to product or service levels. In modern architectures, each domain may define and negotiate intent based on its scope and abstraction level. For example, a transport network may declare an operational intent to "optimize path diversity under bandwidth saturation" — this is an intent, not just a policy. AI agents facilitate the negotiation and refinement of such intents across domains, maintaining semantic consistency and avoiding manual policy stitching [1],[24], [29].

© TM Forum 2025. All Rights Reserved.

Table 2: AI Agent Functional Role

Role	Purpose	Typical TIO Vocabulary
Analysis Agent	Performs KPI correlation, anomaly detection, trend inference, ML-based forecasting.	Consumes met:lastValue, publishes log:Condition evaluation results.
Decision Agent	Selects policies or corrective actions based on business utility, optimization functions, or negotiation outcomes.	Uses mf:logistic, mf:poly mapping functions to score alternatives; outputs revised icm:Intent.
Actuation Agent	Realizes configuration updates in orchestration, controllers, or slice managers.	Invokes TMF Open APIs (TMF641, TMF638) or domain-specific southbound interfaces; reports imo:IntentHandlingEvent.
Learning Agent	Continuously tunes models / thresholds with reinforcement or supervised signals.	Updates policy parameters stored as insp:Template values; may adjust mf:coefficients lists.
Governance Agent	Audits policy conformance, secures ontology/IRI evolution, triggers compliance reporting.	Maintains provenance metadata (prov:wasGeneratedBy if using W3C PROV) and IRI maps under TR289 rules.

- Learning & Adaptation
  - Role - Applying ML techniques to improve agent decision-making over time.

- Functions
  - Uses ML models such as RL, neural networks, or adaptive algorithms.
  - Adapts actions based on closed-loop feedback or environmental changes.
  - Interacts with other agents within a MAS for collective decision-making.
  - Shares state information and synchronizes operations in a distributed system.
  - Negotiates and shares resources, particularly for cross-domain coordination.

© TM Forum 2025. All Rights Reserved.

- Use Case
    - Learning-based Security Agent: Detects anomalies in network traffic using deep learning and adapts security rules over time.
    - ig:IntentNegotiationAgent - Coordinates intent negotiation between different network domains (product, service, and resource layers).
    - Collaborative Network Management Agents - Work in tandem to maintain QoS standards across different regions.
- Communication & Coordination
    - Role - Enabling agents to collaborate and make coordinated decisions.

2.2 Decision-Making Autonomy

- Decision-Making & Reasoning (see Table 3)
    - Role - Using logic, policies, or AI-driven models for autonomous decision-making.
- Functions
    - Applies reasoning models to determine optimal actions based on the available data.
    - Executes decision-making based on predefined rules, AI models (e.g., RL), or adaptive strategies.
    - Executes actions to control the network, such as adjusting configurations, allocating resources, or initiating recovery actions.
    - May trigger workflows or external systems to address performance or security issues.
- Use Case
    - Deliberative Agent - A network resource slicing agent that allocates bandwidth to meet SLA requirements.
    - ig:FaultRecoveryAgent - Identifies network failures, uses logic to classify the issue, and determines the corrective action.
    - Network Optimization Agent - Implements network changes to optimize resource allocation (e.g., rerouting traffic based on current conditions).

© TM Forum 2025. All Rights Reserved.

    - Security Enforcement Agent - Applies security policies to the network, such as activating firewall rules or anomaly detection.

- Action Execution & Control
    - Role - Enacting decisions and performing tasks autonomously.
- AI Agents MAS Intent Base Functional Roles
    - Intent Management Agents
    - ig:IntentExpressionAgent
        - Assembles and submits intent to an intent handler for processing.
    - ig:IntentTranslatorAgent
        - Converts high-level business intent into executable network policies.
    - ig:IntentHandlerAgent
        - Manages the lifecycle of intents, including validation, enforcement, and optimization.
    - ig:IntentMonitoringAgent
        - Observes network/service performance and reports compliance status.
- Network & Resource Management Agents
    - Network Optimization Agent - Dynamically allocates network resources such as bandwidth and routing to improve network performance.
    - ig:FaultRecoveryAgent - Detects network anomalies, triggers corrective actions, and restores service continuity.
    - Security Enforcement Agent - Implements network security policies, such as activating firewalls or detecting network anomalies.
    - Service Quality Agent - Ensures compliance with SLAs and ensures that network services meet QoS parameters.
- AI Agents Autonomy & Decision-Making Capability
    - Reactive Agents (Rule-Based)

© TM Forum 2025. All Rights Reserved.

  - Operate on predefined conditions and respond accordingly (e.g., firewall agent blocking traffic based on static policies).

- Deliberative (Goal-Driven) Agents
  - Perform goal-based reasoning and action. These agents adapt decisions to internal goals or network conditions (e.g., resource slicing based on SLA constraints).

- Learning Agents
  - Use RL or DL to improve decision-making over time, optimizing network performance based on historical data.

- Hybrid Agents
  - Combine rule-based decision-making with ML models for adaptive behavior (e.g., Self-Optimizing AI Agent in telecom networks operations).

Table 3: Decision Making Autonomy

Autonomy Class	Description	TIO-Aligned Examples
Reactive / Static-Policy Agent	Executes fixed rule sets; no model update.	ig:IntentHandlerAgent with hard-coded threshold list (insp:allowedValues).
Deliberative / Goal-Oriented Agent	Plans actions to satisfy high-level goals.	ig:IntentNegotiationAgent running utility optimization using mf:poly.
Learning Agent	Adjusts policies via supervised, RL, or bandit methods.	ig:IntentMonitoringAgent that refines anomaly thresholds with mf:logistic grad-updates.
Hybrid Agent	Combines rule heuristics with learned components.	ig:FaultRecoveryAgent using rule-based triage + ML root-cause scoring.

2.3. AI Agent Interaction Model

- **Autonomous Agents**
  - Operate independently and autonomously without external intervention. These agents perform tasks like self-healing or proactive fault detection.

- **Collaborative Agents in MAS** (see Table 4)
  - Work in coordination to achieve complex goals. These agents cooperate, negotiate, or synchronize operations to manage network resources effectively.

© TM Forum 2025. All Rights Reserved.

- **Hierarchical Agents**
    - Organized in a layered structure, often used for intent propagation across different layers (Product → Service → Resource levels), with each layer having its specific role in fulfilling business and ig:ServiceIntents.

Table 4: Interaction Model

Model	Characteristics	Ontology Links
Autonomous	Minimal coordination; each agent works on local scope.	Agents reference shared met: metrics but no ig:coordinatesWith links.
Collaborative	Agents exchange state via ig:coordinatesWith; may delegate using ig:delegatesTo.	Negotiation & monitoring messages carry imo:Event payloads.
Hierarchical	Upstream owner/handler chain (imo:owner, imo:handler); downstream delegations.	ProductIntent → ServiceIntent → ResourceIntent decomposition via ig:hasSubIntent.

2.4. AI Agent Deployment Scope

- Local Agent (Single-Domain see Table 5)
    - Operate within a single AD, typically focused on a specific function such as resource assurance, access optimization, or local service orchestration. While these agents operate locally, they must coordinate with peers within the domain to avoid conflict, contention, or unstable behaviors. Local autonomy is always scoped by shared goals, domain-wide policy awareness, and agent-to-agent coordination protocols.
    - Local AI agents operate within the boundaries of a single AD, often focused on real-time or near-real-time decision-making for edge, resource, or service orchestration. For these agents to function meaningfully, they must be capable of turning raw data (telemetry, logs, metrics) into contextualized information that informs action.
    - Hence, local agents rely on:
        - Domain-bound ontologies (e.g., YANG models, TMF SID extensions) that define the structure and meaning of local data.
        - Pre-trained or reinforcement-learned models tailored to the local network behavior or service model.
        - Semantic enrichment mechanisms to interpret state in a machine-actionable way.

© TM Forum 2025. All Rights Reserved.

  - Edge AI Agents - Function within edge environments to optimize local resource utilization (e.g., compute, routing, load balancing). However, they cannot operate in isolation—edge agents must coordinate with other local agents to ensure consistency, prevent action loops, and respect shared resource constraints. Uncoordinated decisions may result in systemic instability (e.g., service thrashing or network congestion). Therefore, intra-domain coordination mechanisms—such as local agent consensus, shared state repositories, or intent propagation—are essential to maintain predictable, stable behavior.

Note: Edge agents must balance fast, localized decision-making with collaborative domain behavior. They may leverage domain-level coordinators or shared knowledge graphs to align on global objectives (e.g., avoiding overload or resource starvation). This mirrors practices in distributed systems like SDN controllers, where consistency and coordination models are critical for stability 24, 30.

- Global Agents (Cross-Domain see Table 5)
  - Global agents operate across multiple ADs—e.g., access, core, and cloud—and are responsible for cross-domain intent negotiation, policy harmonization, and resource balancing. These agents must reason over diverse, heterogeneous knowledge base
  - Coordinate operations across multiple network domains, such as across access, transport, core, and cloud layers.
  - Global AI Agents - Facilitate cross-domain intent negotiation, balancing resources across various domains (e.g., access, transport, core, cloud) to ensure global optimization and SLA adherence.
- This is typically achieved through:
  - Ontology alignment or federation, where each domain's ontology is semantically linked to others through mappings or transformation rules.
  - Use of SKOS (Simple Knowledge Organization System) or upper ontologies to define shared vocabularies (e.g., service categories, resource capabilities, QoS classes).
  - Shared conceptual graphs or knowledge hubs that serve as a mediation layer across AI agents operating with different internal models.
- Thus, global agents require (see Table 5):
  - Access to ontology abstraction layers for semantic interoperability.

© TM Forum 2025. All Rights Reserved.

- Mechanisms for semantic translation or reconciliation, such as concept mapping engines or linked data models.

Table 5: Deployment Scop

Scope	Typical Hosting	Example
Local	Single network function or slice control loop.	ig:IntentMonitoringAgent embedded in a gNodeB slice controller.
Domain (AD)	Within an Autonomous Domain.	Product-level ig:IntentExpressionAgent running in Digital BSS.
Cross-Domain	Spans multiple ADs; uses ig:interADNegotiation.	E2E ig:IntentNegotiationAgent brokering latency trade-offs between RAN & Core ADs.
Global	Operator-wide or multi-operator federation.	AI governance agent ensuring TIO IRI compliance (TR289).

© TM Forum 2025. All Rights Reserved.

3. AI Agent MAS Intent-Based Interaction Model

AI agents MAS coordination refers to the mechanisms to enable multiple AI agents to work together to achieve shared or individual goals in a decentralized system. In the context of intent-based ANs, different coordination types ensure effective resource allocation, conflict resolution, and service optimization.

Selecting the appropriate MAS intent-base coordination depends on scalability, optimization objectives, fault tolerance, and decision speed.

A hybrid approach combining hierarchical, distributed, and contract-based coordination is often necessary for robust, real-time AN service management. The selected appropriate coordination would ensure optimal service performance and seamless cross-domain interactions. Below are the primary types of multi-agent coordination.

3.1. Autonomous AI Agents

Autonomous AI Agents operate independently within a defined policy and intent framework, making decisions and executing actions without external intervention. Their intelligence stems not just from detecting failures, but from recognizing non-conformance to desired outcomes — even when systems appear “healthy” from a fault-management perspective.

- Use case, self-healing - AI agent detecting and recovering from failures.
    - A common use case is proactive service assurance in an SD-WAN environment. For example:
        - The network path is operational and fault-free.
        - However, jitter exceeds the customer’s threshold, or latency increases beyond acceptable application tolerance.
        - The AI agent autonomously switches traffic to a lower-cost or higher-performance path that still meets the SLA/OLA — before the user perceives an issue or complains.
        - In this model, nothing is broken, but the system is not aligned with operational or business intent. This represents a shift from failure-driven remediation to intent-driven optimization.
    - Autonomous agents therefore:
        - Continuously monitor for deviations from policy, SLA/OLA, or real-time service metrics.

© TM Forum 2025. All Rights Reserved.

- Use predictive analytics or streaming telemetry to detect degradations before they cause failures.
- Act in proactive cycles, triggering corrective workflows (e.g., policy reapplication, path rerouting, workload reallocation) without waiting for fault alarms or service tickets.

3.2. Collaborative AI Agents

Collaborative AI Agents work together to solve complex tasks through coordination and negotiation.

- Use case, service-level coordination – AI agents negotiating between product, service, and resource layers.

3.3. Hierarchical AI Agents

Hierarchical AI Agents organized in a layered structure for intent propagation.

- Product-Level Agent
  - Defines customer-facing intent.
- Service-Level Agent
  - Converts into service-level requirements.
- Resource-Level Agent (within hierarchical agent)
  - Maps service requirements to network configurations.
  - Resource-level AI agents operate at the infrastructure layer of the hierarchy and are responsible for bridging ig:ServiceIntent with the real-time, fine-grained operational view of the network. Their role extends far beyond issuing configuration commands — they are intelligent observers, interpreters, and synthesizers of the entire operational state.

    Key Functions:
    - Ingest and interpret operational state data across the control, data, and management planes (e.g., telemetry, queues, link status, CPU/memory utilization, interface KPIs).
    - Map service requirements to resource capabilities, not just through static configuration, but through real-time awareness of load, fault domains, topological availability, and failure zones.

© TM Forum 2025. All Rights Reserved.

- Align state data with the underlying schema (e.g., YANG models or TMF resource models) to transform raw telemetry into structured, meaningful information.
- Enrich and correlate localized data with inputs from other agents or domains to form a network-wide semantic view—essential for accurate intent fulfillment, assurance, and self-optimization.
- While configuration change may still be executed these agents primarily operate in a sense-understand-act loop, where the "understand" phase is anchored in data-model-driven reasoning rather than template-driven automation. In modern Tier 1/2 networks, configuration is already handled by policy engines; the agent's value lies in augmenting policy decisions with continuous situational awareness.

3.4. Centralized Coordination Agents

A single coordinating entity (central agent or controller) manages the interactions between AI agents, ensuring optimized decision-making.

- Use cases in Intent-based networks operation:
    - A central intent coordinator agent collects intent requests from multiple ADs and distributes policies to domain-specific agents.
    - Used in hierarchical intent propagation, where higher-level intent managers direct lower-level agents in service and resource orchestration.
- Advantages:
    - Ensures global optimization with full system visibility.
    - Simplifies conflict resolution between agents.
- Disadvantages:
    - Single point of failure.
    - Limits scalability in highly dynamic, large-scale networks.

3.5. Distributed Coordination Agents

Agents coordinate without a central authority, relying on peer-to-peer communication to share information and make decisions.

- Use cases in intent-based networks operations
    - AI Agents in a multi-domain environment negotiate resource allocation through decentralized intent-based negotiations.

© TM Forum 2025. All Rights Reserved.

    - Service-Level Agents adapt dynamically to changes in demand without waiting for higher-level approvals.
- Advantages:
    - Improves scalability and fault tolerance.
    - Allows local optimization and faster decision-making.
- Disadvantages:
    - Requires efficient communication protocols to avoid inconsistencies.
    - Can lead to suboptimal decisions due to limited visibility.

VI. Hierarchical Coordination Agents

A multi-layered coordination approach where AI agents are organized into hierarchical levels, with higher-level agents setting policies for lower-level agents.

Example in Intent-Based Networks:

    - Product-Level AI Agents define high-level business intent.
    - Service-Level AI Agents translate intent into service configurations.
    - Resource-Level AI Agents render network changes via vendor-specific YANG or CLI template/ policy engines. End-to-end declarative intent at this layer is still limited by heterogeneous device models (ongoing IETF NMOP & OpenConfig efforts aim to close the gap)
- Advantages:
    - Ensures structured decision-making across intent layers.
    - Supports intent propagation from high-level business goals to low-level network actions.
- Disadvantages:
    - Increased latency due to multi-layered processing.
    - Requires efficient synchronization between layers.

3.6. Market-Based Coordination AI Agents

Agents act as buyers and sellers in a virtual marketplace, negotiating for resources based on supply and demand.

- Use case in intent-based networks operations:
    - Bandwidth allocation agents bid for network capacity based on service demand.
    - AI-driven ig:IntentNegotiationAgents dynamically price network services based on congestion levels.
- Advantages:

© TM Forum 2025. All Rights Reserved.

    - Provides economic efficiency by aligning supply and demand.
    - Enables dynamic resource allocation in large-scale networks.
- Disadvantages:
    - Requires well-defined pricing and auction mechanisms.
    - Can be computationally intensive.

3.7. Contract-Based Coordination AI Agents

Agents form agreements or smart contracts to ensure compliance with predefined rules and service-level guarantees.

- Use case in intent-based network operations:
    - AI Agents negotiating SLAs in cross-domain services using blockchain-based smart contracts.
    - Ig:Intenthandleragents ensuring policy adherence based on formalized agreements.
- Advantages:
    - Provides transparency and verification.
    - Reduces disputes by formalizing agent commitments.
- Disadvantages:
    - Can introduce complexity in contract enforcement.
    - Requires trust in contract execution mechanisms.

3.8. Coalition-Based Coordination AI Agents

Groups of agents form temporary or long-term coalitions to achieve collective objectives.

- Use cases in intent-based networks operations:
    - Service federation agents coordinate across different telecom operators to ensure seamless roaming and interconnectivity.
    - Security enforcement agents collaborate across multiple domains to detect and mitigate threats.
- Advantages
    - Enhances efficiency in complex multi-agent environments.
    - Enables cooperative resource sharing and policy alignment.
- Disadvantages
    - Requires mechanisms for coalition formation and dissolution.
    - Can lead to conflicts if coalition goals diverge.

© TM Forum 2025. All Rights Reserved.

Game-Theoretic Coordination Agents

Agents make strategic decisions based on mathematical game theory models to optimize their actions in competitive or cooperative settings.

- Use case in intent-based networks operations
    - Ig:AIAgents optimizing network slicing through Nash equilibrium-based strategies.
    - Ig:IntentNegotiationAgents balancing service quality vs. resource costs using multi-agent reinforcement learning.
- Advantages
    - Enables optimal decision-making in competitive environments.
    - Supports both cooperative and adversarial agent interactions.
- Disadvantages
    - Computational complexity increases with agent interactions.
    - Requires accurate modeling of agent preferences and payoffs.

The multi-agent coordination types are shown in Table 6, function coordination type is summarized in Table 7, and Table 8 shows a summary of interaction AI Agent MAS, and use cases for a given application domain.

Table 6: MAS Coordination Type

Coordination Type	Centralized	Distributed	Hierarchical	Market-Based	Contract-Based	Coalition-Based	Game-Theoretic
Scalability	Low	High	Medium	High	Medium	High	Medium
Fault Tolerance	Low	High	Medium	Medium	High	High	Medium
Optimization Focus	Global	Local	Layered	Economic	Compliance	Collaborative	Strategic
Decision Speed	Moderate	High	Medium	High	Medium	Medium	Medium
Implementation Complexity	Medium	High	High	High	Medium	High	High

Table 7: Sample MAS Coordination Type

Category	Function	Coordination Type
Intent Agents	Handle user intent translation & validation	**Negotiation-based**
Optimization Agents	Optimize network, compute, and service resources	**Game-theoretic**
Monitoring Agents	Observe system states & report anomalies	**Distributed RL**
Fault Recovery Agents	Detect and resolve faults autonomously	**Multi-agent RL**
Security Agents	Detect intrusions & enforce security policies	**Multi-agent coordination**

© TM Forum 2025. All Rights Reserved.

Table 8: AI Agent MAS Interaction Use Case and Applications Domain

Type of Interaction	Characteristics	Example Use Case	Applicable Domain
Collaborative (MAS)	Work together to solve complex tasks via coordination and negotiation.	Service Coordination Agents negotiate between product, service, and resource layers.	**All ADs**
Hierarchical Agents	Organized in a structured intent propagation model.	Product-Level Agent defines customer intent → Service-Level Agent converts intent to technical specs → Resource-Level Agent implements network configurations.	**All ADs**
Autonomous Agents	Operate independently, making decisions without external intervention.	Self-Healing Network Agent detects and recovers from failures.	**Service & Resource-Level AD**

3.9. AI Agent MAS Simulation & Inference as a Coordination Mechanism

In an intent-based AN, AI agents may coordinate through simulation and inference, where agents anticipate the state of the network and the likely actions of their peers—not through explicit communication, but through predictive modeling and contextual awareness.

However, the practical realization of this coordination model is currently limited by the way network data is collected and filtered. Today's operational environments often drop or aggregate events at collection points to make the data manageable for human operators. While effective historically, this introduces significant semantic loss—removing the granularity and context required for accurate prediction or inferencing.

For example, important network anomalies may never be recorded in a usable form due to log suppression, low polling frequency, or event deduplication. This directly limits the agent's ability to reason about what has occurred—or to simulate what may happen.

AI agents in an intent-base AN simulate possible network states and infer optimal decisions based on predictive models. This allows agents to coordinate indirectly by anticipating the actions of other agents and making proactive adjustments.

AI agent simulation & inference is an implicit and predictive form of coordination where AI agents anticipate, simulate, and infer actions rather than explicitly communicating with one another. This type of coordination is highly effective in dynamic, complex, and large-scale networks operations where real-time learning

© TM Forum 2025. All Rights Reserved.

and adaptability are critical. It can be used in combination with other coordination mechanisms (e.g., game-theoretic and distributed coordination) to enhance autonomous decision-making, proactive optimization, and self-healing capabilities in intent-based networks operations.

To truly enable simulation and inference:

- Networks must adopt machine-oriented observability architectures, where telemetry, logs, and events are semantically enriched, time-aligned, and retained in full fidelity—even if not all data is immediately acted upon.

- This requires architectural shifts such as:

    - Stream-based telemetry collection (e.g., gNMI, YANG Push)
    - Event correlation layers with semantic tagging
    - Data mesh or fabric models that decouple collection from human interpretability
- Simulation and inference remain a powerful future-state capability for multi-agent coordination—particularly in large-scale, dynamic environments where policy-based coordination is too slow or rigid. However, to be effective, it must be paired with:

    - High-fidelity observability infrastructure
    - Semantic enrichment pipelines
    - Longitudinal data retention for training predictive models
- Without these foundations, simulation may devolve into approximation, and inference into speculation.
- This coordination mode is best seen as complementary to others (e.g., game-theoretic, hierarchical, distributed), and its maturity is tied closely to how well networks evolve to support machine-driven insight rather than human-centric event reduction.

Key Characteristics of Simulation & Inference Coordination

- Predictive Decision-Making

    - AI agents use simulation models (e.g. digital twins, however, a true digital twin of the network layer, with real-time bi-directional fidelity, does not yet exist in current production telecom environments. Monte Carlo simulations) to evaluate potential network scenarios before making real-world decisions. While the term “digital twin” is often used aspirational, in practice, what exists today are model-driven abstractions, partial network simulators, or domain-specific representations (e.g., traffic maps, intent overlays, topology graphs) that support decision-making in constrained scopes.

    - Predictive Decision-Making

© TM Forum 2025. All Rights Reserved.

        - AI agents leverage simulation-assisted inference techniques, including topology models, historical pattern matching, and constrained simulations (e.g., Monte Carlo methods), to evaluate possible outcomes before executing real-world decisions. While full digital twins of the network layer are not yet realizable, agents can operate with partial models or abstracted environment representations to simulate routing, load balancing, or failover scenarios.
        - Use Case: A network optimization agent evaluates alternative routing paths across available infrastructure using policy, telemetry, and topological constraints to infer the most optimal configuration, minimizing service impact without requiring full system reconfiguration.
        - This coordination mechanism is goal-oriented: rather than relying on complete real-time replicas of the network (digital twins), agents work with bounded knowledge graphs, partial topology models, and performance envelopes to reason over likely outcomes and adjust accordingly. Over time, as observability and semantic modeling improve, the fidelity of such simulation will increase — but current architectures still require abstraction and inference rather than twin-like precision.

- Probabilistic Coordination
    - Unlike explicit communication-based coordination (e.g., contract-based or game-theoretic models), simulation & inference rely on probabilistic models to anticipate the behavior of other agents.
    - Use case, traffic prediction agent infers congestion patterns based on historical data and informs load balancing agents to adjust resource allocation.
- Implicit Coordination via Shared Knowledge
    - Instead of directly negotiating, agents infer their actions by referencing a shared knowledge base or intent-driven policies.
    - Use case, ig:IntentNegotiationAgents infer the most likely service demand fluctuations and autonomously adjust their negotiation strategies.
- Cognitive & Learning-Based Coordination
    - AI agents continuously learn from historical events and adapt their decision-making to optimize future coordination.
    - Use case, ig:FaultRecoveryAgent simulates different failure scenarios and proactively deploys mitigation strategies before actual incidents occur.

Simulation & Inference is a non-explicit coordination method where agents act based on internal reasoning or model-based expectation of other agents' behavior. However, inference does not require simulation. It requires

© TM Forum 2025. All Rights Reserved.

interpretable, context-aware information derived from semantically modeled data. Simulation adds value by projecting potential future states, but meaningful inference can occur purely from enriched operational context (e.g., network logs, policy states, KPI trends).

To enable robust inference:

- AI agents must operate on semantically structured data (ontologies, telemetry mapped to models, normalized KPIs).
- Contextual awareness is essential — data must be turned into information, aligned with known schemas, and positioned within the agent's intent or policy domain.

The **AI reasoning pipeline** (Data → Information → Knowledge → Decision). One cannot infer from raw data without semantic grounding. A comparison of Simulation is inference is outlined in Table 9.

Table 9: Comparison of Inference and Simulation

Aspect	Inference	Simulation
Relation	Can occur without simulation	Can enhance inference if model is sound
Purpose	Derive conclusions from known or observed information	Explore outcomes under hypothetical scenarios
Requirement	Semantic understanding of data (information)	Model of system behavior or dynamics

3.10. Self-Healing in Intent-Based AI Agent MAS Service Management

Self-healing is a foundational capability in ANs that enables systems to detect, diagnose, and correct faults without human intervention. In the context of AI Agent MAS operating within an intent-based service management framework, self-healing extends beyond traditional fault remediation to include predictive detection, automated correction, and resilient recovery across the service lifecycle—from order capture to service fulfillment and assurance.

Within the order-to-service provisioning and management platform, self-healing agents continuously monitor workflows, service orchestration chains, and underlying infrastructure states. When anomalies or failures are detected—such as stalled orders, policy violations, or resource allocation mismatches—these agents autonomously trigger predefined or learned corrective actions. These actions may include retrying failed operations, reallocating resources, modifying orchestration paths, or escalating intent modifications to upstream controllers. This proactive remediation reduces the Mean Time to Repair (MTTR), increases service availability, and minimizes manual interventions.

© TM Forum 2025. All Rights Reserved.

AI agents leverage ML models and historical telemetry to identify early warning indicators of service degradation or workflow disruption. This predictive capability is tightly coupled with intent resolution logic, allowing the network to not only react to faults but to anticipate them and act preemptively. For example, if a service intent specifies low-latency delivery for a critical application, and the system detects increasing latency trends along a path, the AI agent can automatically reconfigure the route or reassign workloads to alternative slices or edge nodes.

Furthermore, self-healing is implemented in closed-loop control cycles supported by TMF-compliant APIs, ensuring policy-aligned enforcement and end-to-end visibility. These control loops monitor KPIs and service health indicators in real time, feed insights back into decision policies, and orchestrate corrective actions at both local (intra-domain) and global (cross-domain) levels.

© TM Forum 2025. All Rights Reserved.

4. Cross-Domain Coordination & AD Roles

The ADs are distributed across different layers of the operation stack, including product, service, and resource layers. Within these layers, multiple ADs operate either in parallel or in subordinate hierarchies, each responsible for a specific operational scope within the AN.

AI agent MAS Intent based management within the ADs facilitate dynamic coordination and decision-making. Each AD utilizes AI Agents MAS to process, enforce, and adapt intent dynamically.

AI-based IMF oversee domain-specific responsibilities, ensuring compliance with intent requirements and optimizing performance through a continuous feedback loop. AI agent MAS enable cross-domain collaboration, ensuring effective coordination between intent owners, intent translators, and intent handlers to maintain an optimized network state.

ADs is the vertical layers that deliver and monitor intent, which provides hierarchical responsibility partitions (see Table 10 and Table 11).

Table 10: Cross Domain Coordination using IMF

AD Level (in text)	Primary responsibility	Intent type (ontology)	IM Function role
Product-Level AD	Align with customer offer / CFS	ig:ProductIntent	Owner of downstream intents
Service-Level AD	Translate CFS into service KPIs	ig:ServiceIntent	Both owner *and* handler
Resource-Level AD	Allocate network / compute resources	ig:ResourceIntent	Handler of all upstream goals

Table 11: Primary Intent Type and IMF view

AD Level	Primary Intent Type	IMF View	API Boundary
ProductLevelAD	ig:ProductIntent	Customer promise, commercial SLA	TMF620 TMF622 TMF921
ServiceLevelAD	ig:ServiceIntent	Technical service KPIs, orchestration	TMF633 TMF641 TMF921
ResourceLevelAD	ig:ResourceIntent	Resource allocation, real-time telemetry	TMF639 TMF642

© TM Forum 2025. All Rights Reserved.

Each AD hosts its own Intent Management Function (ig:IntentManagementFunction). Upstream IMF plays **Owner** role (imo:owner) with respect to downstream intents; downstream IMF plays **Handler** role (imo:handler) relative to upstream goals.

Horizontal coordination (ig:interADNegotiation) reconciles trade-offs. Vertical propagation (ig:hasSubIntent) maintains traceability of obligations from Product-level requirements down to resource configurations.

Two relationship statements are expressed in the ontology:

1. Vertical delegation
   "Product-level IMF delegates requirements to the service-level IMF, which in turn delegates to resource-level IMF."
   ⇒ ig:IntentManagementFunction ig:delegatesTo ig:IntentManagementFunction
2. Horizontal negotiation
   "When latency budget must be shared between RAN and Core, the respective IMFs negotiate."
   ⇒ ig:interADNegotiation

AI Agent MAS Interaction Model Across Domains as shown in Figure 2.

- We assume a "horizontal cross-layer structure" that watches over the vertical flow.
- It interfaces with:
  - ig:IntentExpressionAgent IMF to capture business/product goals
  - ig:IntentHandlerAgent to trigger configurations or decisions
  - Ig:IntentMonitoringAgents to detect issues
  - Ig:FaultRecoveryAgent to respond to disruptions
- It may be implemented as a distributed coordination layer or central MAS structure.

© TM Forum 2025. All Rights Reserved.

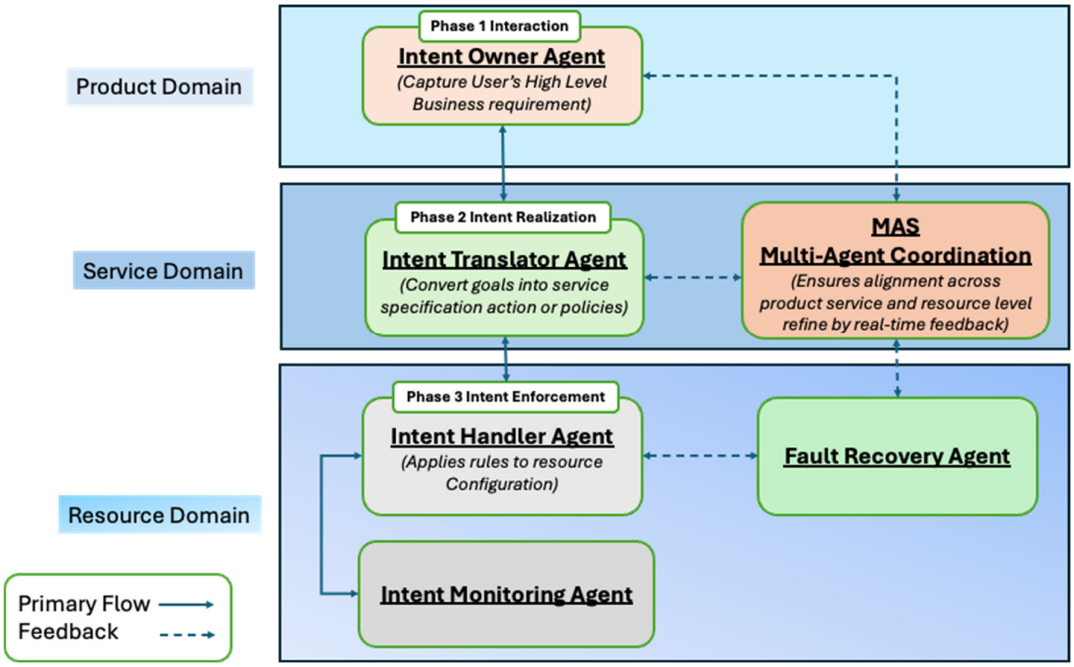

Figure 2: AI Agent MAS Interaction Model across product, service and resource domains

- Agents are mapped clearly to Product, Service, and Resource layers
- Figure 2 displays vertical flow: from Intent Expression (Product layer) → Service decomposition → Execution (Resource layer)
- Multi-Agent Coordinator shown bridging across domains
- Clearer placement of ig:IntentMonitoringAgent and ig:FaultRecoveryAgent

4.1. Multi-Agent System (MAS) Coordination

- The MAS plays a central orchestrating role across the AI agent layers and ADs as follows:
    - Core Responsibilities:
        - Coordinates intent propagation down from Product → Service → Resource layers.
        - Enables policy reconciliation, e.g., ensuring service objectives are consistent across services and resource constraints.
        - Triggers negotiation loops when intent conflicts arise between domains or layers.
        - Supervises execution dependencies (e.g., one service requires another to complete first).
        - May use game-theoretic or rule-based logic for distributed decision-making across domains.

© TM Forum 2025. All Rights Reserved.

4.1.1. MAS-Based Intent Management Across Product, Service, and Resource Domains

- **PHASE 1: Intent Declaration (Product Domain)**
  - Step 1: ig:IntentExpressionAgent
    - Captures high-level business requirements (e.g., customer SLA, service reliability). Expresses the "what" — without specifying the "how".
  - Step 2: ig:IntentTranslatorAgent
    - Converts declarative business goals into machine-readable structured intent, e.g., QoS expectations, delay thresholds, or business policies.
    - Output is sent downward to the Service Domain.

- **PHASE 2 – Intent Realization (Service Domain)**
  - Step 3: ig:IntentHandlerAgent
    - Receives translated goals from Product AD.
    - Converts them into service-specific actions or policies
    - Coordinates with service orchestration systems.
  - Step 4: ig:IntentMonitoringAgent
    - Actively tracks whether service goals are being met
    - Identifies deviations or potential violations.
    - Sends feedback upward to Product Domain
  - Step 5: ig:FaultRecoveryAgent (Service AD)
    - Detects anomalies in service logic
    - May attempt logical recovery, retries, or rerouting requests to alternate services.
  - Step 6: Multi-Agent Coordinator (Service Layer)
    - Orchestrates policy/intent negotiation when goals across domains conflict
    - Bridges Product-Service-Resource interactions to align service delivery with goals.
- **PHASE 3 – Intent Enforcement (Resource Domain)**
  - Step 7: ig:IntentHandlerAgent (Resource AD)
    - Applies service-derived rules into network/device configurations, e.g., activates bandwidth reservation, assigns VLAN tags, provisions edge nodes.
  - Step 8: ig:IntentMonitoringAgent (Resource AD)
    - Monitors real-time network telemetry (e.g., drop rate, jitter, throughput).
    - Detects deviations from intended performance goals.

© TM Forum 2025. All Rights Reserved.

- Step 9: ig:FaultRecoveryAgent (Resource AD)
    - Triggers self-healing procedures
    - Sends observed deviations back to Service Domain

Feedback Loops and Coordination Flows

- Northbound Feedback Arrows (↑):
    - Resource AD → Service AD → Product AD
    - Used when performance drops or failures occur.
- Southbound Intent Flow (↓):
    - Product AD → Service AD → Resource AD
    - Used for goal propagation and configuration delegation.
- Cross-Layer Coordination (↔):
    - Handled by the Multi-Agent Coordinator, ensuring all agents remain aligned with global objectives, KPIs, and operational states.

© TM Forum 2025. All Rights Reserved.

5. Intent Lifecycle & MAS Coordination

The intent lifecycle is structured into three high-level phases—Discovery, Negotiation, Fulfillment—that describe *how an expressed intent moves from user goal to operational enforcement* across Product, Service, and Resource domains. Each phase activates one or more AI agents (ig:AIAgent subclasses) that pass structured intent, validate feasibility, reconcile conflicts, and ultimately drive execution. Lifecycle transitions are captured in the ontology using ig:initiatesPhase, ig:delegatesTo, ig:coordinatesWith, and ig:resolvesIntent. Feedback loops (compliance, degradations, recovery) use monitoring and fault agents to re-enter upstream phases when conditions drift, as summarized in Table 12.

Table 12: Intent Lifecycle

Phase	Trigger
Discovery	ig:IntentExpressionAgent asserts new icm:Intent; Owner set (imo:owner).
Negotiation	ig:IntentNegotiationAgent evaluates feasibility; posts imo:IntentHandlingEvent (imo:IntentAccepted / imo:IntentRejected).
Fulfillment	ig:IntentTranslationAgent and ig:IntentHandlerAgent translate & enact; Monitoring updates compliance via imo:Complies, imo:Degrades.

5.1. TIO Lifecycle Concepts

Intent Expression vs Fulfillment. Intent is a *declarative outcome statement* (what, not how). Expression occurs when a user or system declares goals (e.g., low latency, high reliability). Fulfillment occurs when those goals are translated into actions and configurations. TIO models the intent as an icm:Intent instance; domain-specific forms (Product/Service/Resource) are represented here as ig:ProductIntent, ig:ServiceIntent, and ig:ResourceIntent (all subclasses of icm:Intent).

Lifecycle Phases as Context Markers. Figure 3 use *phase boxes* (Discovery, Negotiation, Fulfillment) to show where agents act; these are modeled as instances of ig:IntentLifecyclePhase subclasses (ig:DiscoveryPhase, ig:NegotiationPhase, ig:FulfillmentPhase). They are *organizational constructs*—not computational objects—but linking agents to phases improves traceability and tooling.

© TM Forum 2025. All Rights Reserved.

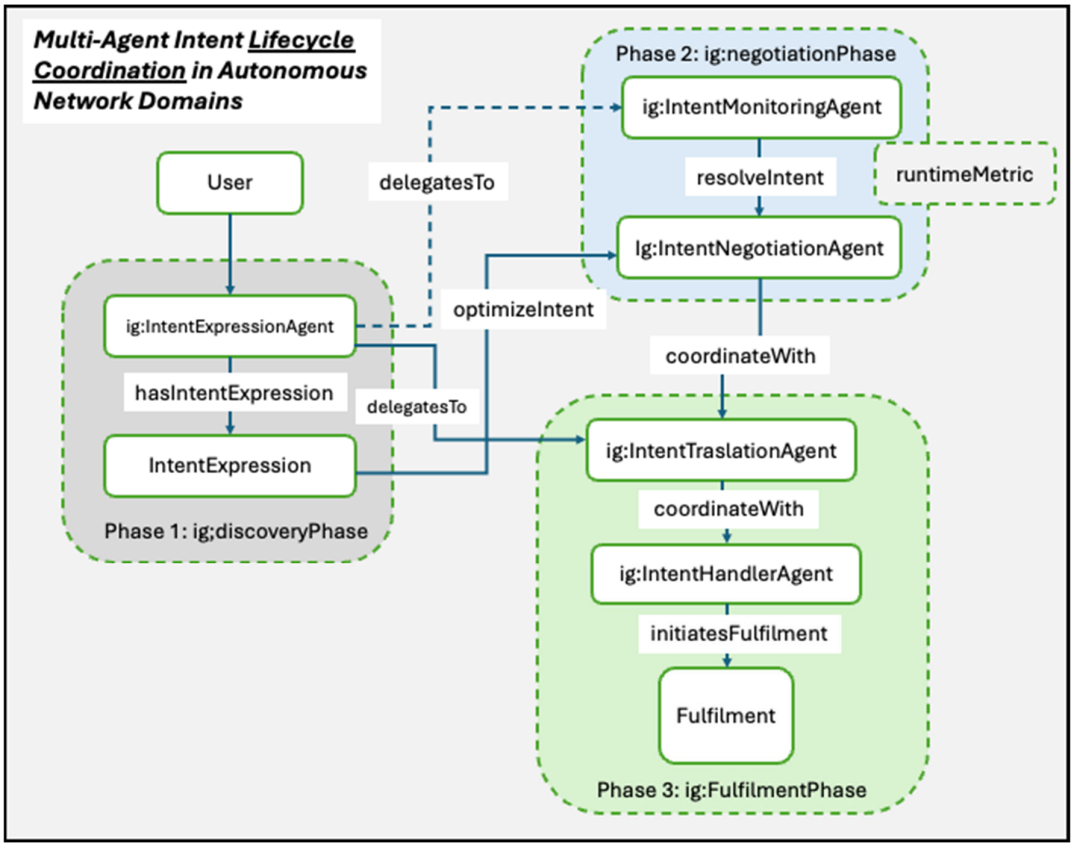

Figure 3: Lifecycle Phases

5.2. Phase 1 - Discovery

Initiated by: ig:IntentExpressionAgent.
Primary Goal: Capture a user/business requirement and formalize it as an icm:Intent instance (often a ig:ProductIntent in product scope).
Key Outputs: Structured intent, semantic annotations, scope (locations, service class, KPI targets), and optional priority.

5.2.1. Semantic Capture

The IntentExpressionAgent receives natural language or structured input and produces an icm:Intent description. During semantic capture, it may reference catalog elements (via TMF620 Product Catalog [4], TMF622 Product Order [5]) to bind the intent to a product offer or CFS. This binding is recorded with ig:hasIntentExpression.

5.2.2. Lifecycle Trigger

Once a valid intent structure exists, the agent asserts:
ig:IntentExpressionAgent ig:initiatesPhase ig:DiscoveryPhase .
This signals that downstream validation can begin (monitoring pre-checks, feasibility screens).

© TM Forum 2025. All Rights Reserved.

5.2.3. Monitoring Activation

At discovery close, the ig:IntentExpressionAgent triggers the ig:IntentMonitoringAgent so feasibility and baseline compliance checks can start—even *before* provisioning. This supports proactive detection of unachievable targets (e.g., minimum achievable latency).

Phase 2 — Negotiation

Entered when: Discovery output must be reconciled against policy, resource constraints, cost, or cross-domain service conditions.
Agents: ig:IntentNegotiationAgent (lead), coordinating with ig:IntentExpressionAgent, ig:IntentHandlerAgent, and ig:IntentMonitoringAgent for evidence.

5.2.4. Feasibility & Trade-off Resolution

The Negotiation Agent evaluates whether declared targets (latency, bandwidth, uptime) are *achievable* under current network and policy constraints. When multiple domains (Product, Service, Resource) are involved, it requests domain-specific feasibility via TMF APIs (e.g., TMF641 Service Order qualification flows, TMF645 Service Qualification, TMF921 Intent status queries).
Negotiation results may include:

- Adjusted numeric thresholds (e.g., 25ms target revised to 40ms acceptable).
- Policy substitutions (alternate service class).

These adjustments are represented as new or updated icm:Intent instances linked via ig:optimizesIntent or as subordinate intents linked by ig:hasSubIntent.

5.2.5. Multi-Agent Coordination

Negotiation models is based on *horizontal* exchanges:
IntentNegotiationAgent ig:coordinatesWith ig:IntentExpressionAgent .
IntentNegotiationAgent ig:coordinatesWith ig:IntentHandlerAgent .
IntentNegotiationAgent ig:delegatesTo ig:IntentHandlerAgent .

Where cross-domain alignment is needed:
ig:interADNegotiation is asserted between the Intent Management Functions of the participating ADs (Product ↔ Service ↔ Resource).

5.2.6. Lifecycle Transition

When negotiation converges, the Negotiation Agent marks:
ig:IntentNegotiationAgent ig:initiatesPhase ig:NegotiationPhase .
Using TIO event semantics, completion (accepted, modified, rejected) is signaled through status events (e.g., imo:IntentAccepted, imo:IntentRejected).

© TM Forum 2025. All Rights Reserved.

5.3. Phase 3 — Fulfillment

Entered when: Negotiated intent is ready to be implemented.
Agents: ig:IntentTranslationAgent (maps to operations), ig:IntentHandlerAgent (executes), plus domain orchestrators.

5.3.1. Intent Translation

The IntentTranslationAgent converts the negotiated, semantically structured intent into *actionable* service or resource directives. Typical transformations include:

- Mapping business KPIs to service policy parameters.
- Selecting service templates (CFS → RFS decomposition).
- Inserting technology-specific variables (bandwidth floor, slice class).

This agent coordinates with catalog APIs (TMF620, TMF633), service order APIs (TMF641), and resource-level APIs (TMF642 / TMF639) depending on depth of decomposition. ig:delegatesTo is used when translation must hand control to a Handler in a downstream AD.

5.3.2. Intent Handling / Execution

The IntentHandlerAgent receives translated parameters and interacts with orchestration stacks, controllers, or provider systems to enact them. Enforcement may be declarative (policy expressions) or imperative (device configs). Handler execution aligns with TMF641 (Service Order), TMF638/639 (Inventory), or domain-specific controllers.

5.2.3. Monitoring During Fulfillment

The Monitoring Agent continues to observe telemetry during and after fulfillment. When measured values drift outside declared (or renegotiated) ranges, it asserts a relationship such as: ig:IntentMonitoringAgent ig:resolvesIntent <theIntent> (resolution attempt), possibly re-entering Negotiation or triggering Fault Recovery.

5.3.4. Fault Recovery

If the Handler cannot meet requirements or the service degrades, the FaultRecoveryAgent attempts remediation (retry, alternate path, partial rollback). It coordinates with Handler and Monitoring Agents and may surface escalations upstream (Product AD).

5.4. Feedback Flows

Three directional feedback patterns are modeled:

© TM Forum 2025. All Rights Reserved.

Table 13: Feedback Flows

	Path	Used When	Ontology Links
Southbound Intent Flow (↓)	Product → Service → Resource	Goal propagation and decomposition.	ig:hasSubIntent, ig:delegatesTo, TMF620→TMF641 chains.
Northbound Performance Feedback (↑)	Resource → Service → Product	Compliance, degradation, failure.	ig:resolvesIntent, ig:optimizesIntent, TMF921 intent status.
Cross-Layer Coordination (↔)	Peer ADs	Conflict resolution, policy arbitration.	ig:interADNegotiation, ig:coordinatesWith.

These flows allow upstream layers to see when downstream enforcement fails or degrades and allow downstream layers to request goal relaxation or cost trade-offs.

5.5. Ontology Lifecycle

The ontology lifecycle expresses lifecycle semantics using lightweight structural links rather than procedural state machines as shown in Table 14: Ontology Lifecycle turtle syntax. Core relationships:

Table 14: Ontology Lifecycle turtle syntax

Ontology Lifecycle Turtle syntax
Discovery ig:IntExprAgent1 a ig:IntentExpressionAgent ; ig:hasIntentExpression ex:Intent_a; ig:initiatesPhase ig:DiscoveryPhase ; ig:coordinatesWith ig:IntMonAgent1 . # Negotiation ig:IntNegAgent1 a ig:IntentNegotiationAgent ; ig:coordinatesWith ig:IntExprAgent1 , ig:IntHandlerAgent1 ; ig:optimizesIntent ex:Intent_a ; ig:initiatesPhase ig:NegotiationPhase . # Fulfillment ig:IntTransAgent1 a ig:IntentTranslationAgent ; ig:delegatesTo ig:IntHandlerAgent1 ; ig:initiatesPhase ig:FulfillmentPhase .

© TM Forum 2025. All Rights Reserved.

```
ig:IntHandlerAgent1 a ig:IntentHandlerAgent ;
  ig:resolvesIntent ex:Intent_a .
```

5.6. TIO Lifecycle Events

Track lifecycle transitions in event streams (Kafka, TMF notification topics, etc.), link the ontology to TIO's event families by annotating agent actions with imo:IntentHandlingEvent subtypes:

- imo:IntentRegistered
- imo:IntentAccepted
- imo:IntentRejected
- imo:IntentComplies
- imo:IntentDegrades
- imo:IntentRemediated

These events don't replace the structural ig: properties; they provide runtime state and history. You can associate events to phases via annotations or SHACL rules (e.g., "Discovery complete when Accepted or Rejected").

© TM Forum 2025. All Rights Reserved.

6. AI Agent MAS and Intent Management Function (IMF)

IMF agents facilitate intent lifecycle management, ensuring translation, execution, and monitoring.

ig:IntentManagementFunction (IMF) instances supervise the lifecycle of every icm:Intent, orchestrating translation, execution, monitoring, and optimization through a federation of AI agents (ig:AIAgent subclasses).

6.1. AI Agents and Network & Resource Management

AI agents perform real-time optimization, fault recovery, security enforcement, and SLA assurance. They fall into four autonomy classes:

Table 15: Autonomy Classes

Class	Characteristic	Example
Reactive (rule-based)	Static policy lookup	ACL Firewall Agent (Service / Resource AD)
Deliberative (goal-driven)	Symbolic reasoning	Network-Slicing Agent reallocating spectrum (Product / Service AD)
Learning (ML / RL)	Reinforcement adaptation	RL Traffic-Prediction Agent (Service / Resource AD)
Hybrid	Rules + ML	Self-Optimizing Agent combining fixed thresholds and learned patterns

6.1.1. Interaction model for the MAS

- Intra-domain coordination – agents inside one ig:AutonomousDomain collaborate via ig:coordinatesWith.
- Inter-domain negotiation – upstream and downstream IMFs exchange proposals by asserting ig:interADNegotiation links to peer IMFs.

6.2. Intent-Based MAS Framework across AD Layers

Table 16: Autonomy Layer Domain

AD Layer	Key AI-Agent Role	Typical Function
Product-Level AD (ig:ProductLevelAD)	Intent-Owner Agent	Define commercial / CFS goals
Service-Level AD (ig:ServiceLevelAD)	Intent-Translation & Monitoring Agents	Convert business intent to service policies; track KPIs

© TM Forum 2025. All Rights Reserved.

AD Layer	Key AI-Agent Role	Typical Function
Resource-Level AD (ig:ResourceLevelAD)	Intent-Handler & Fault-Recovery Agents	Enforce configurations; remediate failures

6.2.1. TIO Sample MAS Use-Case table

Table 17: TIO Use Cases

Agent (TIO class)	Function	Example	AD Scope
ig:NetworkOptimizationAgent	Dynamic resource allocation	Reroute to avoid congestion	Service & Resource
ig:FaultRecoveryAgent	Detect & heal failures	MPLS FRR < 50 ms; CPE dual-homing	Service & Resource
ig:SecurityEnforcementAgent	Apply adaptive security	Detect lateral movement → micro-segmentation	Service & Resource
ig:ServiceQualityAgent	Proactive SLA governance	Adjust slice bandwidth to meet latency	Service

6.3. Knowledge-Graph Representation of Intent State

The runtime **A-Box** graph consists of:

- **Nodes** – AI agents (ig:AIAgent), intents (icm:Intent), metrics (met:RuntimeMetric), network elements.
- **Edges** – delegation (ig:hasSubIntent), coordination (ig:coordinatesWith), optimisation (ig:optimizesIntent).
- **Attributes** – compliance flags, timestamps, utility scores.

Validation uses SHACL shapes (e.g., ig:IntentShape) to guarantee each ig:IntentMonitoringAgent links to at least one met:RuntimeMetric.

Partial-graph reports are SHACL-conformant subsets aligned with TMF TR290B; they contain only the entities relevant to the intent under observation and a temporal window for validity.

6.3.1. Example monitoring report

- Compliance – KPIs met / breached
- Observed metrics – latency, bandwidth, fault count
- Affected entities – device X, slice Y

© TM Forum 2025. All Rights Reserved.

- Temporal window – t:hasBeginning "2026-08-20T00:00Z" … t:hasEnd "2030-09-20T00:05Z"

6.4. AI-Agent Roles in Graph Updates (A-Box)

Table 18: AI Agent Role

Role	Triple-store activity	Note
Intent Owner Agent (ig:IntentExpressionAgent)	Create / update intent instances	No T-Box edits
Intent Handler Agent (ig:IntentHandlerAgent)	Map intent → policies; write compliance status	A-Box only
Intent Monitoring Agent (ig:IntentMonitoringAgent)	Insert real-time met:RuntimeMetric values	A-Box only
Intent Negotiation Agent (ig:IntentNegotiationAgent)	Update intent after cross-domain negotiation	A-Box; links via ig:resolvesIntent

6.5. Semantic intent vs. operational execution

- Intent expression layer – declares the *what* (icm:Intent).
- Analytics / policy layer – evaluates goals using thresholds, mf:UtilityFunction, and predictive models.
- Actuation layer – enforces via TMF APIs (ig:APIInstance → TMF641, TMF642, etc.).

This separation aligns with TMF IG1230 and TR290A/B: declarative goals remain technology-agnostic while execution logic is programmable and vendor-specific.

6.6. Key Roles in the Intent Pipeline

Figure 4 shows the semantic representation of intent, which remains technology-agnostic, while the execution logic resides **in** programmable modules tuned to the operator's architecture, data sources, and performance goals.

This aligned with TMF IG1230, where the transformation of declarative business objectives into automated enforcement must traverse multiple AI agent layers and data pipelines.

- Intent Expression Layer: Defines the "what"—e.g., latency should be below **X** ms, packet loss below **Y%**, bandwidth utilization above **Z%**.
- Analytics & Policy Engines: Interpret semantic expressions, perform threshold comparisons, statistical evaluations, predictive analysis, or conflict resolution to determine actionable decisions.

© TM Forum 2025. All Rights Reserved.

- Actuation Layer: Enforces changes through configuration updates, control plane signaling, or orchestration mechanisms.

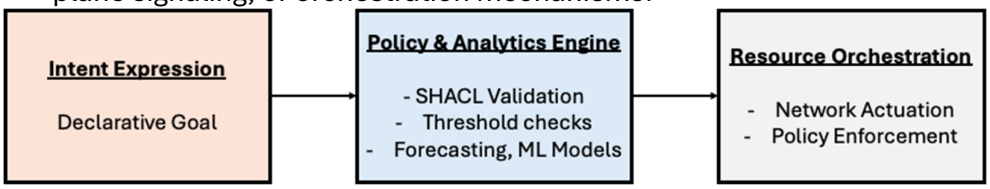

Figure 4: Intent-to-Action Pipeline

6.6.1. Bridging Declarative Intent and Operational Execution

The flow **Figure 5** illustrates the semantic pipeline for realizing intent in an autonomous service management system. It begins with Intent Expression, where user or system-defined objectives (e.g., performance, policy, reliability) are declared in a structured and technology-agnostic format. These are passed to the Semantic Parsing layer, which translates and maps the intent into formal representations aligned with domain ontologies and policy models. The Analytics Engine then interprets these parsed intents by evaluating current network state, telemetry, and performance metrics to determine whether the goals are achievable or need adaptation. Finally, the pipeline culminates in Action, where AI agents or orchestrators execute configuration changes, trigger workflows, or initiate optimization mechanisms based on the resolved intent. This flow underscores the decoupling of declarative goals from operational execution.

Figure 5: Semantic Execution of Network Intent

TMF TR290 [36] and its sub-models (A, B) formally define the expression and reporting structures that feed into the semantic parsing and analytics layers.

© TM Forum 2025. All Rights Reserved.

7. Intent-Driven Autonomous Domains using TIO

The OWL ontology describes how intent, AI agents, and ADs interact in an autonomous-network environment. All class and property names below use the TIO family of prefixes as shown in Table 19.

Table 19: TIO Prefixes

Prefix	Purpose
`ig:`	Extension (agents, ADs, lifecycle, API)
`icm:`	Intent Common Model (abstract intent object)
`mf:`	Math / utility functions
`met:`	Metrics & observations
`log:`	Negotiation artefacts

7.1. Ontology representation components

The AD ontology components and responsibility are detailed in Table 20 and Table 21.

Table 20: Ontology Components

Concept	TIO class / property	Comment
Autonomous Domain (abstract)	ig:AutonomousDomain	Container for closed-loop control at each layer.
Product-Level AD	ig:ProductLevelAD	Handles commercial goals, maps to CFS.
Service-Level AD	ig:ServiceLevelAD	Turns CFS into service KPIs / RFS.
Resource-Level AD	ig:ResourceLevelAD	Orchestrates resources to realize RFS.
IntentManagementFunction (IMF)	ig:IntentManagementFunction	One per AD; linked via ig:hasIMF.
AI agents	subclasses of ig:AIAgent	Linked to each AD via ig:hasAgent.
Utility Function	mf:UtilityFunction	Used by optimisation agents.
Runtime Metric	met:RuntimeMetric	KPI sample / observation objects.

© TM Forum 2025. All Rights Reserved.

Table 21: Autonomous Domain Responsibility

AD responsibility	Ontology element	Key link
Commercial intent → CFS	ig:ProductLevelAD	ig:hasProductIntent
CFS → Service-KPI / RFS mapping	ig:ServiceLevelAD	ig:hasServiceIntent
RFS orchestration	ig:ResourceLevelAD	ig:hasResourceIntent

7.1.1. Intent flow across ADs

- ig:ProductIntent originates in ig:ProductLevelAD (e.g. "Premium service, < 40 ms").
- ig:ServiceLevelAD refines it into a service intent (ig:ServiceIntent) – *network slice QoS, traffic-class, etc.*
- ig:ResourceLevelAD executes the resulting resource intent (ig:ResourceIntent).
- Delegation is recorded with ig:hasSubIntent and ig:delegatesTo.
- Intent never attaches to the abstract ig:AutonomousDomain class—only to a concrete AD instance.

Table 22: Intent flow across ADs

Property	Domain → Range	Purpose
ig:hasIMF	ig:AutonomousDomain → ig:IntentManagementFunction	Each AD's closed-loop control.
ig:hasAgent	ig:AutonomousDomain → ig:AIAgent	Places agents inside an AD.
ig:intraADNegotiation	IMF → IMF	Conflict resolution within the same AD.
ig:interADNegotiation	IMF → IMF	Cross-domain negotiation.

7.1.2. TIO IMF properties

- Interpretation / execution / negotiation – inherent role of each ig:IntentManagementFunction.
- Intent propagation – ig:hasProductIntent, ig:hasServiceIntent, ig:hasResourceIntent.
- Optimisation – ig:optimizesIntent (IMF or agent → intent).
- Utility linkage – ig:hasUtility (agent → mf:UtilityFunction).
- Decomposition – ig:hasSubIntent (intent → intent).
- IMF-to-IMF negotiation – ig:interADNegotiation.

These properties implement the same governance principle for every layer; the Service-level IMF typically provides end-to-end coordination by mediating between Product and Resource IMFs.

© TM Forum 2025. All Rights Reserved.

8. Intent Specification & Template Mapping

Intent expressions captured in the Discovery phase must be transformed into machine-actionable policy fragments so that orchestration and assurance systems can enforce them. Use Intent Specification (TIO **insp:**) vocabulary as defined in TMF TR299—to bind goals *to* parameter sets, allowed values, and conditional logic.

Intent expressions captured in the Discovery phase as:

- an Intent object (class icm:Intent);
- a set of parameters (target values, units, thresholds); and
- an optional template reference that tells downstream functions which fields are required.

A template represents the business policy schema against which the intent is interpreted.

8.1. Parameter Representation

Table 23: Parameter Representation

Field	Ontology term in TIO alignment	Example in text
parameter	insp:parameter	oneWayLatency
unit	insp:unit (links to quan:)	quan:millisecond
allowedValues / range	insp:allowedValues	1..40
mandatory flag	insp:mandatory "true"^^xsd:boolean	—

8.2. Declarative vs Operational Layers

Table 24: Declarative vs. Operation

Layer	Purpose	Core Classes
Semantic Expression	Abstract goals: *"latency < 40 ms between source and destination."*	icm:Intent, ig:ProductIntent, ig:ServiceIntent, ig:ResourceIntent
Specification	Formal parameter slots, units, value ranges.	insp:Specification, insp:Template, insp:allowedValues, insp:unit
Operational Data	Runtime metric series & state snapshots.	met:Metric, met:Observation, met:lastValue

© TM Forum 2025. All Rights Reserved.

Layer	Purpose	Core Classes
Validation & Events	Compliance checks, state transitions.	log:Condition, imo:IntentHandlingEvent

Using this stack keeps vendor-specific enforcement logic (mf: math functions, set filters, etc.) isolated from the intent's *declarative semantics* (see Table 24).

8.2.1. Template Example

Table 25: A Template Example

A Template
ex:LowLatencyTemplate a insp:Template ; insp:parameter "oneWayLatency" ; insp:unit quan:millisecond ; insp:allowedValues "1..40"^^xsd:string ; insp:mandatory true .

When the ig:IntentExpressionAgent creates ex:Intent_a, it references this template:

Table 26: Template

Template
ex:Intent_a icm:usesTemplate ex:LowLatencyTemplate ; icm:targetValue "35"^^xsd:integer .

Later, the Monitoring Agent evaluates compliance by comparing met:lastValue against the template's insp:allowedValues.

8.3. API Integration (TMF Open APIs)

The concrete API instances that carry specification data are modeled with ig:APIInstance:

Table 27: API Integration

AD Layer	APIInstance Role	API Spec
ProductLevelAD	Expose product catalog & ordering, capture product intent.	ig:TMF620ProductCatalogAPI, ig:TMF622ProductOrderAPI, ig:TMF921IntentManagementAPI
ServiceLevelAD	Exchange service orders, qualify resources, post ig:ServiceIntent.	ig:TMF633ServiceCatalogAPI, ig:TMF641ServiceOrderAPI, ig:TMF645ServiceQualificationAPI, ig:TMF921IntentManagementAPI

© TM Forum 2025. All Rights Reserved.

AD Layer	APIInstance Role	API Spec
ResourceLevelAD	Allocate resources & update inventory.	ig:TMF639ResourceInventoryAPI, ig:TMF642ResourceOrderAPI

Table 28: Linkage Pattern

Linkage Pattern
ig:IntMgmtAPI_Global a ig:APIInstance ; ig:conformsToAPI ig:TMF921IntentManagementAPI ; ig:bindsDomain ig:ProductLevelAD ; ig:endpointURI <https://prod.example.com/tmf-api/intent/v1> .

APIInstance triples allow tooling to verify that *every* domain boundary exposes the endpoints required by lifecycle flows.

8.4. Validation & SHACL Hooks

SHACL is considered as a lightweight conformance layer. The core NodeShape (ig:AIAgentShape) already appears in the TTL; here are two additional shapes.

8.4.1. IntentComplianceShape (excerpt)

These shapes enforce that every intent references at least one template, and every APIInstance advertises a valid HTTPS endpoint and is linked to at least one AD. Implementers can extend with stricter checks (e.g., compare insp:allowedValues to quan: units).

Table 29: Intent Compliance Shape

Intent Compliance Shape
ig:IntentComplianceShape a sh:NodeShape ; sh:targetClass icm:Intent ; sh:property [sh:path icm:usesTemplate ; sh:minCount 1 ;] ; sh:property [sh:path icm:targetValue ; sh:datatype xsd:decimal ;] .

© TM Forum 2025. All Rights Reserved.

8.4.2. APIBoundaryShape (excerpt)

Table 30: API Boundary Shape

API Boundary Shape
ig:APIBoundaryShape a sh:NodeShape ; sh:targetClass ig:APIInstance ; sh:property [sh:path ig:endpointURI ; sh:pattern "^https?://" ;] ; sh:property [sh:path ig:bindsDomain ; sh:minCount 1 ;] .

8.4.3. Multi-Agent System (MAS) Coordination

- The MAS plays a central orchestrating role across the AI agent layers and ADs as follows:
  - Core Responsibilities:
    - Coordinates intent propagation down from Product → Service → Resource layers.

SHACL NodeShape Validation for AIAgent
For an AIAgent within a MAS, it's important to validate the data that describes the agent to ensure it:

- Complies with predefined structures and constraints: This helps ensure that the agent behaves predictably within the system.
- Avoids incorrect interactions: For example, preventing an IntentExpressAgent from being connected to an invalid ig:IntentMonitoringAgent.
- Provides confidence in agent interactions: Ensuring that agents can interact safely without violating data constraints helps guarantee that the MAS works efficiently.

Use Case - Consider a situation where you have multiple AI agents working in a telecom network system. Each agent needs to:

- Handle specific tasks (e.g., intent expression, monitoring, negotiation).
- Interact with other agents to complete tasks.

SHACL NodeShape validation:

© TM Forum 2025. All Rights Reserved.

- These agents are involved in reasoning, transformation, or monitoring of intents

MAS Lifecycle Coordination - Ontology

Figure 6 and the Turtle listing in Table 31 contain the following TIO-based elements:

- Lifecycle-phase model – the class ig:IntentLifecyclePhase with concrete instances ig:DiscoveryPhase, ig:NegotiationPhase, and ig:FulfillmentPhase.
- MAS-coordination properties – ig:coordinatesWith, ig:delegatesTo, ig:initiatesPhase, ig:resolvesIntent – used to express horizontal collaboration, vertical hand-off, phase triggers, and compliance resolution.
- OWL class restrictions – each AI-agent specialization carries owl:someValuesFrom constraints on ig:hasIntentExpression (→ icm:Intent) and ig:hasRuntimeMetric (→ met:RuntimeMetric).
- SHACL validation – the node shape ig:AIAgentShape ensures every ig:AIAgent instance links to at least one intent and one runtime metric.
- Programmatic discoverability – key classes and properties include rdfs:seeAlso pointers to their related TM Forum Open-API specifications (e.g., ig:TMF921IntentManagementAPI), enabling automated tooling to resolve documentation and interface details.

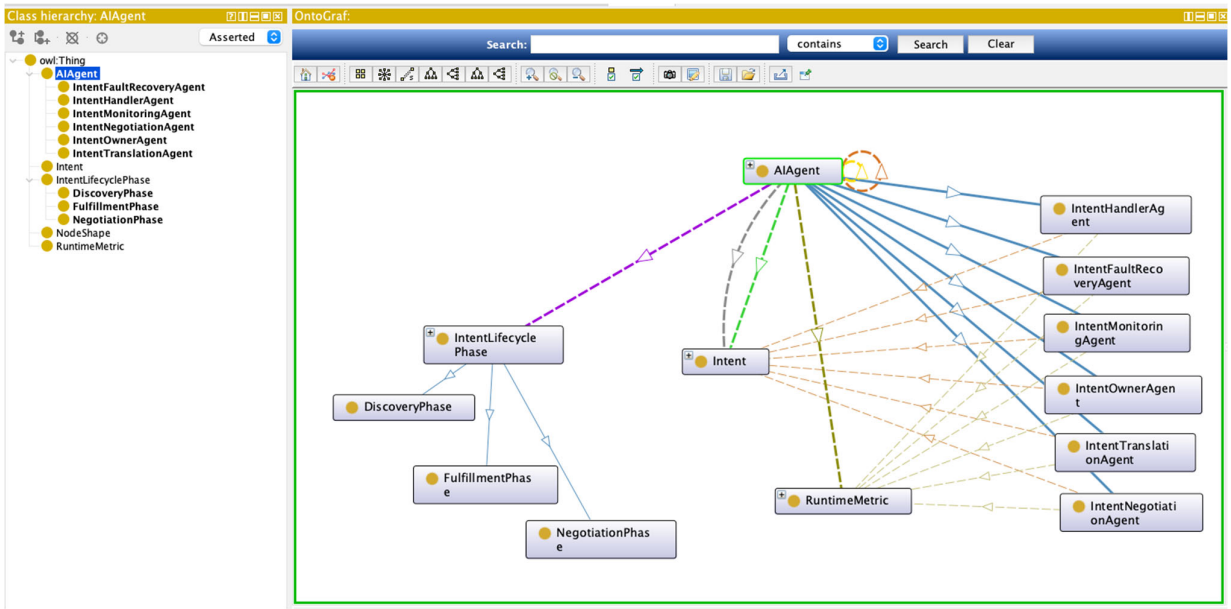

Figure 6: TIO MAS Protege Class Hierarchy

© TM Forum 2025. All Rights Reserved.

Table 31: MAS Class Hierarchy Turtle Syntax

MAS class hierarchy TM Forum TIO prefixes
@base <http://tio.models.tmforum.org/ig1421/v1.1.0/> . @prefix ig: <http://tio.models.tmforum.org/ig1421/v1.1.0/> . @prefix icm: <http://tio.models.tmforum.org/tio/v3.6.0/IntentCommonModel/> . @prefix met: <http://tio.models.tmforum.org/tio/v3.6.0/MetricsAndObservations/> . @prefix sh: <http://www.w3.org/ns/shacl#> . @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> . @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> . @prefix owl: <http://www.w3.org/2002/07/owl#> . @prefix xsd: <http://www.w3.org/2001/XMLSchema#> . # Core superclass ig:AIAgent a owl:Class ; rdfs:seeAlso ig:AIAgentShape . # Lifecycle phases ig:IntentLifecyclePhase a owl:Class . ig:DiscoveryPhase a owl:Class ; rdfs:subClassOf ig:IntentLifecyclePhase . ig:NegotiationPhase a owl:Class ; rdfs:subClassOf ig:IntentLifecyclePhase . ig:FulfillmentPhase a owl:Class ; rdfs:subClassOf ig:IntentLifecyclePhase . # Intent & metric classes icm:Intent a owl:Class . # from the Intent-Common-Model met:RuntimeMetric a owl:Class . # from Metrics & Observations # Agent specialisations ig:IntentFaultRecoveryAgent a owl:Class ; rdfs:subClassOf ig:AIAgent , [a owl:Restriction ; owl:onProperty ig:hasRuntimeMetric ; owl:someValuesFrom met:RuntimeMetric] , [a owl:Restriction ; owl:onProperty ig:hasIntentExpression ; owl:someValuesFrom icm:Intent] . ig:IntentHandlerAgent a owl:Class ; rdfs:subClassOf ig:AIAgent , [a owl:Restriction ; owl:onProperty ig:hasIntentExpression ; owl:someValuesFrom icm:Intent] , [a owl:Restriction ; owl:onProperty ig:hasRuntimeMetric ; owl:someValuesFrom met:RuntimeMetric] . ig:IntentMonitoringAgent a owl:Class ; rdfs:subClassOf ig:AIAgent , [a owl:Restriction ; owl:onProperty ig:hasRuntimeMetric ; owl:someValuesFrom met:RuntimeMetric] , [a owl:Restriction ; owl:onProperty ig:hasIntentExpression ; owl:someValuesFrom icm:Intent] . ig:IntentNegotiationAgent a owl:Class ; rdfs:subClassOf ig:AIAgent , [a owl:Restriction ; owl:onProperty ig:hasIntentExpression ; owl:someValuesFrom icm:Intent] , [a owl:Restriction ; owl:onProperty ig:hasRuntimeMetric ; owl:someValuesFrom met:RuntimeMetric] . ig:IntentOwnerAgent a owl:Class ; rdfs:subClassOf ig:AIAgent , [a owl:Restriction ; owl:onProperty ig:hasRuntimeMetric ; owl:someValuesFrom met:RuntimeMetric] , [a owl:Restriction ; owl:onProperty ig:hasIntentExpression ; owl:someValuesFrom icm:Intent] . ig:IntentTranslationAgent a owl:Class ; rdfs:subClassOf ig:AIAgent , [a owl:Restriction ; owl:onProperty ig:hasIntentExpression ; owl:someValuesFrom icm:Intent] ,

© TM Forum 2025. All Rights Reserved.

```
  [ a owl:Restriction ; owl:onProperty ig:hasRuntimeMetric    ;
owl:someValuesFrom met:RuntimeMetric ] .

# Object- and datatype-properties
ig:hasIntentExpression a owl:ObjectProperty ;
  rdfs:domain ig:AIAgent ; rdfs:range icm:Intent .

ig:hasRuntimeMetric a owl:ObjectProperty ;
  rdfs:domain ig:AIAgent ; rdfs:range met:RuntimeMetric .

ig:initiatesPhase a owl:ObjectProperty ;
  rdfs:domain ig:AIAgent ; rdfs:range ig:IntentLifecyclePhase .

ig:coordinatesWith a owl:ObjectProperty ;
  rdfs:domain ig:AIAgent ; rdfs:range ig:AIAgent .

ig:delegatesTo a owl:ObjectProperty ;
  rdfs:domain ig:AIAgent ; rdfs:range ig:AIAgent .

ig:resolvesIntent a owl:ObjectProperty ;
  rdfs:domain ig:AIAgent ; rdfs:range icm:Intent .

# SHACL shape to validate agents
ig:AIAgentShape a sh:NodeShape ;
  sh:targetClass ig:AIAgent ;
  sh:property [
     sh:path ig:hasIntentExpression ;
     sh:class icm:Intent ;
     sh:minCount 1
  ] ;
  sh:property [
     sh:path ig:hasRuntimeMetric ;
     sh:class met:RuntimeMetric ;
     sh:minCount 1
  ] .
```

© TM Forum 2025. All Rights Reserved.

9. IMF and AD – Agentic-AI Protégé Ontology – TIO

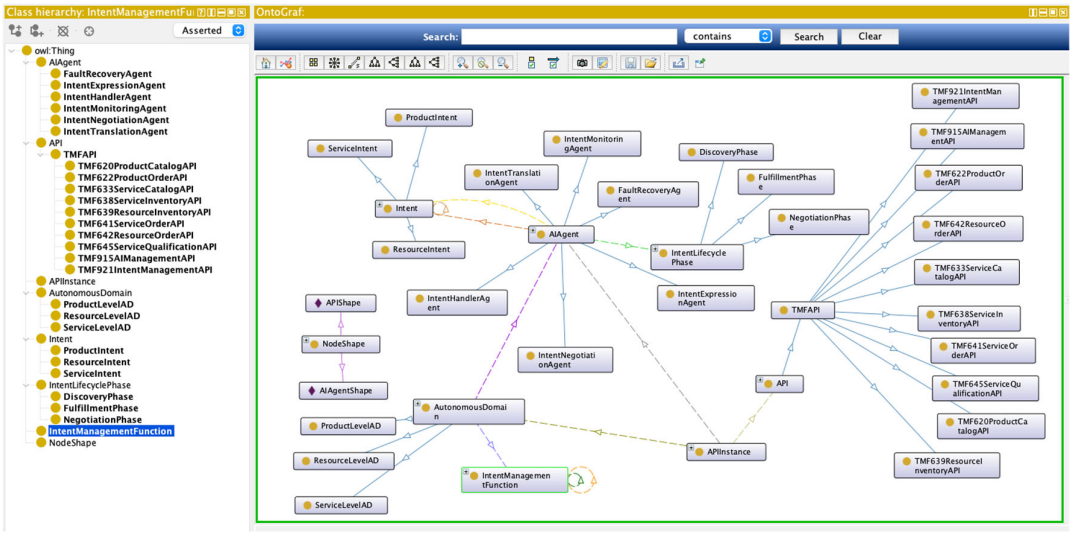

Figure 7: TIO Protégé IMF for AD – Agentic-AI MAS Ontology

Figure **7** illustrates the complete TIO-aligned ontology for intent-driven automation. It shows every ig:APIInstance bound to its TMF Open-API specification, the three AD ig:ProductLevelAD, ig:ServiceLevelAD, ig:ResourceLevelAD—their closed-loop ig:IntentManagementFunction objects, the intent-centric AI agents, the lifecycle-phase instances (ig:DiscoveryPhase, ig:NegotiationPhase, ig:FulfillmentPhase), utility functions, and the negotiation artefacts that link IMFs across domains.

The ontology offers an OWL vocabulary that supports intent-based, AI-agent-orchestrated service management in autonomous telecom domains by capturing:

- **Entities** – ig:AIAgent subclasses, icm:Intent instances (ig:ProductIntent, ig:ServiceIntent, ig:ResourceIntent), ig:AutonomousDomain, ig:IntentManagementFunction, lifecycle phases, mf: utility functions, log:NegotiationSession, and telemetry/metric objects (met:Metric, met:Observation).
- **Relationships** – downward intent propagation (ig:hasSubIntent, ig:delegatesTo) from Product → Service → Resource; upward feedback (ig:resolvesIntent, ig:optimizesIntent); horizontal coordination & trade-off resolution (ig:coordinatesWith, ig:interADNegotiation); API bindings (ig:bindsDomain, ig:invokedByAgent, ig:conformsToAPI).
- **Constraints & validation** – SHACL shapes such as ig:AIAgentShape and ig:APIShape ensure compliance (e.g., every ig:IntentMonitoringAgent must reference at least one met:Metric and every ig:APIInstance must specify an HTTPS ig:endpointURI).

© TM Forum 2025. All Rights Reserved.

This TIO-aligned model provides a machine-readable foundation for tooling, reasoning, and cross-domain interoperability in autonomous-network service management.

Table 32: Ontology Prefix and Namespace

Prefix	Namespace IRI	Notes / Module Origin
ig:	http://tio.models.tmforum.org/ig1421/v1.1.0/	IG1421-specific extension: AI-agent subclasses, AD hierarchy, lifecycle phases, API vocabulary.
imo:	http://tio.models.tmforum.org/tio/v3.6.0/IntentManagementOntology#	TIO Intent-Management Ontology (IMF classes, intent-handling events).
icm:	http://tio.models.tmforum.org/tio/v3.6.0/IntentCommonModel/	Common intent object model (declarative goal expression).
insp:	http://tio.models.tmforum.org/tio/v3.6.0/IntentSpecification/	Parameter templates, allowed-value sets, units.
met:	http://tio.models.tmforum.org/tio/v3.6.0/MetricsAndObservations/	Runtime metrics, observations, KPI samples.
mf:	http://tio.models.tmforum.org/tio/v3.6.0/MathFunctions/	Utility / cost functions, mathematical operators.
quan:	http://tio.models.tmforum.org/tio/v3.6.0/QuantityOntology#	Units of measure (ms, %, Mbps, etc.).
log:	http://tio.models.tmforum.org/tio/v3.6.0/LogicalOperators#	Logical / conditional expressions (AND, OR, NOT, IF-THEN).
set:	http://tio.models.tmforum.org/tio/v3.6.0/SetOperators/	Set / list operators (union, intersection, difference).
sec:	http://tio.models.tmforum.org/tio/v3.6.0/SecurityOntology#	Security concepts
secmet:	http://tio.models.tmforum.org/tio/v3.6.0/SecurityMetrics/	Security-specific metric vocabulary.
rdf, rdfs, owl, xsd, sh:	standard W3C namespaces	Core RDF, RDFS, OWL, XML-Schema, SHACL vocabularies.

Core Class Hierarchy

```
ig:AIAgent
 ├─ ig:IntentExpressionAgent
 ├─ ig:IntentMonitoringAgent
 ├─ ig:IntentNegotiationAgent
 ├─ ig:IntentTranslationAgent
 └─ ig:IntentHandlerAgent
```

© TM Forum 2025. All Rights Reserved.

```
icm:Intent          (abstract parent from the Intent Common Model)
 ├─ ig:ProductIntent
 ├─ ig:ServiceIntent
 └─ ig:ResourceIntent

ig:IntentLifecyclePhase
 ├─ ig:DiscoveryPhase
 ├─ ig:NegotiationPhase
 └─ ig:FulfillmentPhase

ig:AutonomousDomain
 ├─ ig:ProductLevelAD
 ├─ ig:ServiceLevelAD
 └─ ig:ResourceLevelAD

ig:IntentManagementFunction        (per-domain IMF)

mf:UtilityFunction              (math/utility functions, TIO Math-Functions module)
met:RuntimeMetric               (runtime metric / observation, TIO Metrics module)
log:NegotiationSession          (logical negotiation context, TIO Logical-Operators
module)
```

Table 33: Key object properties

TIO-aligned Property	Domain → Range	Purpose / Comment
ig:hasIntentExpression	ig:AIAgent → icm:Intent	Links an intent-producing agent (Expression, Negotiation, etc.) to the intent object it declares.
ig:initiatesPhase	ig:AIAgent → ig:IntentLifecyclePhase	Marks the point at which an agent triggers **Discovery**, **Negotiation**, or **Fulfillment**.
ig:coordinatesWith	ig:AIAgent ↔ ig:AIAgent	Horizontal collaboration / peer interaction.
ig:delegatesTo	ig:AIAgent → ig:AIAgent	Vertical hand-off (e.g., Translation → Handler).
ig:resolvesIntent	ig:IntentMonitoringAgent → (icm:Intent | met:RuntimeMetric)	Compliance validation or metric-based resolution.
ig:hasSubIntent	icm:Intent → icm:Intent	Hierarchical decomposition (Product → Service → Resource).
ig:optimizesIntent	ig:AIAgent → icm:Intent	Links an optimisation or utility-selection activity to the affected intent.
ig:hasAgent	ig:AutonomousDomain → ig:AIAgent	Places one or more agents inside an AD.

© TM Forum 2025. All Rights Reserved.

TIO-aligned Property	Domain → Range	Purpose / Comment
ig:hasIMF	ig:AutonomousDomain → ig:IntentManagementFunction	Associates each AD with its closed-loop IMF.
ig:interADNegotiation	ig:IntentManagementFunction → ig:IntentManagementFunction	Negotiation / conflict resolution **across** domain boundaries.
ig:intraADNegotiation	ig:IntentManagementFunction → ig:IntentManagementFunction	Negotiation **within** a single AD.

TIO SHACL validation layer

- ig:AIAgentShape (sh:NodeShape)
  Targets every ig:AIAgent instance and checks that required capability links—such as ig:hasIntentExpression, ig:coordinatesWith, or ig:invokedByAgent—are present.
- ig:IntentShape (sh:NodeShape)
  Applies to each **icm:Intent** (including ig:ProductIntent, ig:ServiceIntent, ig:ResourceIntent).
  Validates that:
  - a priority value is provided (icm:priority),
  - at least one target metric is referenced (met:targetMetric), and
  - an explicit temporal window or validity period is defined (t:hasBeginning / t:hasEnd).
- ig:NegotiationOutcomeShape (sh:NodeShape)
  Targets every negotiation outcome node (instance of log:NegotiationSession or the event class imo:IntentAccepted / imo:IntentRejected).
  Ensures it contains exactly one link back to the originating intent expression (ig:hasIntentExpression) and, if modified, exactly one reference to the updated intent (ig:optimizesIntent).

These SHACL shapes constitute the validation layer that guarantees conformance of all runtime data to the TIO.

Reasoning & usage scenarios — TIO

1. Lifecycle alignment
   Using SWRL or SPARQL CONSTRUCT rules, the system infers that a proposed **ig:ServiceIntent** must enter ig:NegotiationPhase when the triple ig:hasConflict "true"^^xsd:boolean is present.
2. Cross-domain queries
   A SPARQL query can ask: *"Which ig:ResourceIntents roll up to a given product intent?"* by traversing the transitive path ig:hasSubIntent* from an ig:ProductIntent down to all matching ig:ResourceIntent instances.
3. Compliance dashboards
   An ig:IntentMonitoringAgent publishes met:RuntimeMetric instances.

© TM Forum 2025. All Rights Reserved.

SHACL shapes mark non-conforming triples, and dashboards render the graph state as red/green compliance indicators.

4. Automated negotiation
   ig:IntentNegotiationAgent instances consume negotiation artefacts (e.g. log:NegotiationSession, imo:IntentRejected) and output revised intents linked via ig:resolvesIntent to the original goal.
5. Closed-loop optimisation
   Utility/cost models stored as mf:UtilityFunction objects allow agents to rank alternative service variants and choose the option that maximizes declared utility while satisfying constraints.

All rules, queries, and shapes operate over the TIO-aligned namespaces (ig:, icm:, met:, mf:, log:), ensuring consistent reasoning across autonomous domains.

TIO TM Forum references

Table 34: TMF Alignment

TM Forum artefact	TIO-aligned element in this ontology	Alignment note
IG1253 Intent Management Function (IMF)	ig:IntentManagementFunction	Class directly represents the IMF defined in IG1253; each ig:AutonomousDomain asserts one instance via ig:hasIMF.
eTOM layered architecture – Product / Service / Resource	ig:ProductLevelAD, ig:ServiceLevelAD, ig:ResourceLevelAD (sub-classes of ig:AutonomousDomain)	Domain layering in the ontology mirrors the eTOM stack, allowing intent delegation (ig:hasSubIntent, ig:delegatesTo) through the same levels.
TMF Open APIsTMF620 Product CatalogTMF633 Service CatalogTMF921 Intent Management	ig:TMF620ProductCatalogAPI, ig:TMF633ServiceCatalogAPI, ig:TMF921IntentManagementAPI (sub-classes of ig:TMFAPI) plus their deployed ig:APIInstance objects	Classes such as ig:ProductIntent or ig:IntentHandlerAgent include rdfs:seeAlso links to the relevant API specs, indicating which TMF interface is typically used to carry that data (*e.g.*, rdfs:seeAlso ig:TMF921IntentManagementAPI).

These mappings ensure that every ontology element can be traced back to its authoritative TM Forum reference, enabling consistent implementation across standards-based systems.

© TM Forum 2025. All Rights Reserved.

Turtle File for MAS Agentic-AI Protégé Ontology

Table 35: Turtle syntax of the complete Ontology

```
@base <http://tio.models.tmforum.org/ig1421/v1.1.0/> .
@prefix ig:     <http://tio.models.tmforum.org/ig1421/v1.1.0/> .
@prefix imo:    <http://tio.models.tmforum.org/tio/v3.6.0/IntentManagementOntology#>
.
@prefix icm:    <http://tio.models.tmforum.org/tio/v3.6.0/IntentCommonModel/> .
@prefix insp:   <http://tio.models.tmforum.org/tio/v3.6.0/IntentSpecification/> .
@prefix log:    <http://tio.models.tmforum.org/tio/v3.6.0/LogicalOperators#> .
@prefix set:    <http://tio.models.tmforum.org/tio/v3.6.0/SetOperators/> .
@prefix met:    <http://tio.models.tmforum.org/tio/v3.6.0/MetricsAndObservations/> .
@prefix mf:     <http://tio.models.tmforum.org/tio/v3.6.0/MathFunctions/> .
@prefix quan:   <http://tio.models.tmforum.org/tio/v3.6.0/QuantityOntology#> .
@prefix sec:    <http://tio.models.tmforum.org/tio/v3.6.0/SecurityOntology#> .
@prefix secmet:  <http://tio.models.tmforum.org/tio/v3.6.0/SecurityMetrics/> .
@prefix t:      <http://www.w3.org/2006/time#> .
@prefix rdf:    <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs:   <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl:    <http://www.w3.org/2002/07/owl#> .
@prefix xsd:    <http://www.w3.org/2001/XMLSchema#> .
@prefix sh:     <http://www.w3.org/ns/shacl#> .

# TIO Ontology
<> a owl:Ontology ;
  rdfs:comment "Ontology aligned to TM Forum Intent Ontology (TIO). Includes lifecycle
phases & TMF API vocabulary."@en .

# 1  AI-Agent Hierarchy
ig:AIAgent              a owl:Class .
ig:IntentExpressionAgent   a owl:Class ; rdfs:subClassOf ig:AIAgent .
ig:IntentMonitoringAgent   a owl:Class ; rdfs:subClassOf ig:AIAgent .
ig:IntentNegotiationAgent  a owl:Class ; rdfs:subClassOf ig:AIAgent .
ig:IntentTranslationAgent  a owl:Class ; rdfs:subClassOf ig:AIAgent .
ig:IntentHandlerAgent      a owl:Class ; rdfs:subClassOf ig:AIAgent .
ig:FaultRecoveryAgent      a owl:Class ; rdfs:subClassOf ig:AIAgent .
```

© TM Forum 2025. All Rights Reserved.

```
# 2  Lifecycle Phases  (added)
ig:IntentLifecyclePhase a owl:Class .
ig:DiscoveryPhase      a owl:Class ; rdfs:subClassOf ig:IntentLifecyclePhase .
ig:NegotiationPhase    a owl:Class ; rdfs:subClassOf ig:IntentLifecyclePhase .
ig:FulfillmentPhase    a owl:Class ; rdfs:subClassOf ig:IntentLifecyclePhase .

# 3  Intent Specialisations
ig:ProductIntent   a owl:Class ; rdfs:subClassOf icm:Intent .
ig:ServiceIntent   a owl:Class ; rdfs:subClassOf icm:Intent .
ig:ResourceIntent  a owl:Class ; rdfs:subClassOf icm:Intent .

# 4  Autonomous Domains & IMF
ig:AutonomousDomain a owl:Class .
ig:ProductLevelAD   a owl:Class ; rdfs:subClassOf ig:AutonomousDomain .
ig:ServiceLevelAD   a owl:Class ; rdfs:subClassOf ig:AutonomousDomain .
ig:ResourceLevelAD  a owl:Class ; rdfs:subClassOf ig:AutonomousDomain .

ig:IntentManagementFunction a owl:Class ;
  rdfs:seeAlso imo:IntentManager .

# 5  Object & Datatype Properties

ig:hasAgent        a owl:ObjectProperty ; rdfs:domain ig:AutonomousDomain ; rdfs:range
ig:AIAgent .
ig:hasIMF          a owl:ObjectProperty ; rdfs:domain ig:AutonomousDomain ; rdfs:range
ig:IntentManagementFunction .
ig:hasProductIntent   a owl:ObjectProperty .
ig:hasServiceIntent   a owl:ObjectProperty .
ig:hasResourceIntent  a owl:ObjectProperty .

ig:delegatesTo       a owl:ObjectProperty .
ig:coordinatesWith    a owl:ObjectProperty .
ig:interADNegotiation a owl:ObjectProperty ; rdfs:domain ig:IntentManagementFunction ;
rdfs:range ig:IntentManagementFunction .
ig:intraADNegotiation a owl:ObjectProperty ; rdfs:domain ig:IntentManagementFunction ;
rdfs:range ig:IntentManagementFunction .
```

© TM Forum 2025. All Rights Reserved.

```
ig:hasIntentExpression a owl:ObjectProperty ; rdfs:domain ig:AIAgent ; rdfs:range icm:Intent .

ig:initiatesPhase     a owl:ObjectProperty ; rdfs:domain ig:AIAgent ; rdfs:range ig:IntentLifecyclePhase .

ig:resolvesIntent     a owl:ObjectProperty .

ig:hasSubIntent       a owl:ObjectProperty ; rdfs:domain icm:Intent ; rdfs:range icm:Intent .

ig:optimizesIntent    a owl:ObjectProperty ; rdfs:domain ig:AIAgent ; rdfs:range icm:Intent .


# 6  TMF Open-API Vocabulary

ig:API a owl:Class .

ig:TMFAPI a owl:Class ; rdfs:subClassOf ig:API .


ig:TMF620ProductCatalogAPI      a owl:Class ; rdfs:subClassOf ig:TMFAPI .

ig:TMF622ProductOrderAPI        a owl:Class ; rdfs:subClassOf ig:TMFAPI .

ig:TMF633ServiceCatalogAPI      a owl:Class ; rdfs:subClassOf ig:TMFAPI .

ig:TMF638ServiceInventoryAPI    a owl:Class ; rdfs:subClassOf ig:TMFAPI .

ig:TMF641ServiceOrderAPI        a owl:Class ; rdfs:subClassOf ig:TMFAPI .

ig:TMF639ResourceInventoryAPI   a owl:Class ; rdfs:subClassOf ig:TMFAPI .

ig:TMF642ResourceOrderAPI       a owl:Class ; rdfs:subClassOf ig:TMFAPI .

ig:TMF645ServiceQualificationAPI a owl:Class ; rdfs:subClassOf ig:TMFAPI .

ig:TMF915AIManagementAPI        a owl:Class ; rdfs:subClassOf ig:TMFAPI .

ig:TMF921IntentManagementAPI    a owl:Class ; rdfs:subClassOf ig:TMFAPI .


ig:APIInstance a owl:Class .


ig:conformsToAPI a owl:ObjectProperty ; rdfs:domain ig:APIInstance ; rdfs:range ig:API .

ig:bindsDomain   a owl:ObjectProperty ; rdfs:domain ig:APIInstance ; rdfs:range ig:AutonomousDomain .

ig:invokedByAgent a owl:ObjectProperty ; rdfs:domain ig:APIInstance ; rdfs:range ig:AIAgent .

ig:endpointURI   a owl:DatatypeProperty ; rdfs:domain ig:APIInstance ; rdfs:range xsd:anyURI .

ig:apiVersion    a owl:DatatypeProperty ; rdfs:domain ig:API ; rdfs:range xsd:string .

ig:specDocument   a owl:DatatypeProperty ; rdfs:domain ig:API ; rdfs:range xsd:anyURI .


# 7  Minimal SHACL Shapes
```

© TM Forum 2025. All Rights Reserved.

```
ig:AIAgentShape a sh:NodeShape ;
  sh:targetClass ig:AIAgent .

ig:APIShape a sh:NodeShape ;
  sh:targetClass ig:APIInstance ;
  sh:property [ sh:path ig:endpointURI   ; sh:pattern '^https?://' ] ;
  sh:property [ sh:path ig:conformsToAPI ; sh:minCount 1 ] .

# End of File
```

© TM Forum 2025. All Rights Reserved.

10. Summary

This guide sets out the first end-to-end blueprint that maps TM Forum's Intent Ontology (TIO) directly onto an operational architecture for Autonomous Networks (ANs).

Table 36: Summary of the Main Contribution

Pillar	Main contribution
Ontology alignment	Extends TIO with an ig: namespace (agents, life-cycle phases, IMF/AD links, API classes). The resulting OWL/Turtle model captures how Product-, Service-, and Resource-level intents are decomposed, negotiated and enforced by specialized AI-agents inside each AD.
Lifecycle model	Formalizes a three-phase loop – Discovery → Negotiation → Fulfilment – and binds each phase to agent roles (ig:IntentExpressionAgent, ig:IntentNegotiationAgent, etc.) plus runtime events (imo:) and SHACL validation.
MAS taxonomy & coordination	Classifies agents by function, autonomy, interaction mode, deployment scope; surveys seven coordination styles (hierarchical, distributed, market-based ...) and shows how they are expressed with ig:coordinatesWith, ig:delegatesTo, ig:interADNegotiation.
Intent ⇄ API bridge	Introduces ig:APIInstance so that every TMF Open APIs, becomes a semantic object; this lets agents discover, invoke and version-negotiate service-management endpoints programmatically.
Closed-loop analytics	Couples' intents to metrics (met:RuntimeMetric) and utility functions (mf:) so monitoring agents can publish compliance graphs and feed optimizers – enabling proactive assurance and self-healing.
Cross-domain IMF governance	Shows how Intent-Management Functions in separate ADs act as owners or handlers of intents, negotiate trade-offs, and propagate delivery expectations downward via ig:hasSubIntent.
Worked examples	Provides tables, Turtle snippets and sequence diagrams for service latency negotiation, RL-based optimization, and KPI breach remediation.

This guide introduces Multi-Agent System (MAS) as follows:

1. Layer-aligned delegation – A dedicated MAS runs inside each AD:
    - ig:ProductLevelAD hosts Expression & Owner agents,
    - ig:ServiceLevelAD hosts Translation, Monitoring & Negotiation agents,
    - ig:ResourceLevelAD hosts Handler & Fault-Recovery agents.
      Via ig:delegatesTo and ig:hasSubIntent, intents are decomposed and handed down the stack while feedback bubbles upward.
2. Horizontal collaboration – Within a given AD, peer agents coordinate through ig:coordinatesWith to share telemetry, schedule actions, and avoid race conditions (e.g., optimisation vs. fault recovery).

© TM Forum 2025. All Rights Reserved.

3. Cross-domain negotiation – ig:IntentNegotiationAgent instances, governed by their IMFs, exchange proposals across AD boundaries using ig:interADNegotiation, allowing slice latency, bandwidth, and cost trade-offs to be resolved autonomously.
4. Closed-loop optimisation – MAS agents jointly implement the MADE loop (Monitor-Analyze-Decide-Execute) by publishing met:RuntimeMetric, computing mf:UtilityFunction scores, and invoking TMF APIs through typed ig:APIInstances.

10.1. Key MAS Parts

- Formal role taxonomy – Codifies six agent classes plus negotiation/optimisation patterns so vendors can plug-in specialized agents without breaking the ontology.
- Semantic coordination fabric – Introduces explicit object-properties (ig:coordinatesWith, ig:delegatesTo, negotiation links) that let reasoning engines *see* and verify multi-agent workflows.
- API-native execution – Binds every agent action to discoverable TMF API calls, turning MAS plans into deterministic, audited service-management transactions.
- Inter-AD governance model – Defines how IMFs supervise their local agent swarms while still participating in operator-wide, game-theoretic negotiations—bridging local autonomy and global KPIs coherence.

10.2. Conclusion & Outlook

Key finding — Intent semantics alone are insufficient; they must be married to an agentic execution layer, explicit life-cycle events, and machine-navigable APIs.

This guide demonstrates that when TIO concepts (Intent, Metric, Utility) are linked to role-specialized AI-agents and TMF APIs, operators can turn product-level promises into real-time, closed-loop control across heterogeneous domains without manual policy stitching.

Finally, this guide provides MAS from an implementation detail to a first-class, ontology-driven control plane, proving that TM-Forum-aligned semantics can orchestrate dozens of cooperating agents while remaining interoperable, auditable, and API-centric.

By operationalizing TIO through an agent-centric, API-aware framework, the guide provides self-optimizing, standards-aligned Autonomous Networks. The next wave of work should focus on (i) machine-discoverable API registries, (ii) high-fidelity observability for simulation, and (iii) governance patterns that keep multi-agent swarms safe, auditable, and interoperable across vendors and operators.

© TM Forum 2025. All Rights Reserved.

10.2.1. Forward-looking prospects

Table 37: Opportunities and Standards Gaps

Opportunity	Research / standard gaps
API-first agent gateways that expose self-discoverable affordances.	Standard JSON-LD vocabulary for action pre/post-conditions, automatic capability negotiation.
Causal simulation & partial digital twins wired into the Negotiation phase so agents can run *what-if* checks before enforcing policies.	Lightweight, vendor-neutral twin schema; performance-safe "shadow execution" zones at scale.
Cross-operator IMF federation – sharing intent graphs and SLA proofs to enable roaming, slicing and green-routing across footprints.	Secure provenance exchange (W3C PROV + verifiable credentials) and multi-party policy arbitration.
Ontology-driven DevOps – catalog updates, network changes and AI-model retraining all described in the same semantic layer, enabling full supply-chain traceability.	Tooling to co-evolve ontologies, code and models; automated regression tests for SHACL shapes & policies.
AI-native governance & audit – every agent call, negotiation artefact and policy change logged as semantic events, feeding compliance dashboards and LLM-based explainers.	Unified event schema (imo:) plus formal verification of agent plans against safety/o-policy constraints

© TM Forum 2025. All Rights Reserved.

11. Appendix A: MAS Agentic-AI Intent-Based AN Operations – Formulation and Analytics

The ig:AIAgent intent-based ANs enabled to dynamically interpret, enforce, and adapt to evolving services and resources real-time demands. Unlike traditional SLOs, where requirements remain static and do not propagate to existing instances, intent-driven operations use AI-agent intent-based IMF to ensure real-time compliance and adaptation.

AI agents, such as the ig:IntentExpressionAgent, continuously refine and update intent requirements, ensuring that any changes immediately affect already deployed service and resource instances. This level of automation enhances network efficiency by allowing ig:IntentTranslatorAgents to convert high-level business goals into executable policies and ig:intenthandleragents to manage the full intent lifecycle, including validation, enforcement, and optimization.

SLOs are traditionally managed by human operators, whereas intent-based AI-driven systems allow AI agents to autonomously negotiate and enforce new requirements dynamically. These ig:IntentMonitoringAgents track network compliance and generate intent reports, ensuring alignment between network states and business goals. The shift to ig:AIAgent intent-based operations marks a critical advancement in network automation, transforming static management paradigms into intelligent, self-optimizing frameworks.

11.1. Agentic-AI MAS AN

Autonomous networks are structured as ADs, as defined by the TMF IG1253. These ADs are distributed across various layers of the network operation stack—product, service, and resource layers. Within each layer, multiple *ig:AIAgent* work collaboratively, forming a MAS that ensures compliance with intent-driven requirements. In this hierarchical framework, ADs leverage intent management agents to coordinate inter and intra-domain interactions. Each AI Agent within a domain has a well-defined role, such as optimizing resources, enforcing security policies, or ensuring service quality. This architecture follows a structured approach:

- Product-Level ig:AIAgent
    - Manage customer-facing services, ensuring business intent alignment.
- Service-Level ig:AIAgent

© TM Forum 2025. All Rights Reserved.

    - Convert business requirements into actionable service-level configurations.
- Resource-Level ig:AIAgent
    - Execute network orchestration by configuring and optimizing infrastructure resources.

Intent-based AI agent coordination enables an adaptive network where each domain sets and enforces requirements using standardized APIs. This process ensures seamless interaction between domains, fostering an intelligent and self-sustaining network ecosystem.

11.2. Agentic-AI Enabled Intent Management

Each AD integrates Intent Managers, IMF, realized through AI agent intent based cognitive loops, to coordinate intent-based operations. These IMF communicate using intent statements, ensuring network goals are met efficiently within AI agent different roles. The AI agent intent-based IMF leverage APIs to manage the entire lifecycle of intent.

ig:TMF921IntentManagementAPI enables communication between ig:AIAgent, facilitates intent registration, status tracking, and compliance monitoring. This API enables AI-driven automation by allowing systems to modify requirements dynamically, ensuring real-time adaptation to changing network conditions. ig:AIAgent utilize ig:TMF921IntentManagementAPIs to

- Efficiently manage intent-based automation without modifying operator-defined SLOs. ig:TMF921IntentManagementAPI capabilities
- Modifies intent dynamically to optimize service delivery.
- Monitors compliance with both SLOs and intent-driven network policies.
- Handles feasibility checks to determine network capability in meeting intent-driven requirements.
- Facilitates inter-domain negotiations to optimize service and resource allocation.
- Continuously assess the network's compliance with intent-based objectives
- Ensures seamless interaction between Intent Owners and Intent Handlers
- Modifies intent to reflect evolving business needs.
- Reporting compliance status and identifies optimization opportunities.
- Handles feasibility checks to determine network capability in meeting intent-driven requirements.
- Facilitating inter-domain negotiations, seamless intent propagation.

© TM Forum 2025. All Rights Reserved.

11.3. AD Hierarchy Model

The hierarchy of ADs includes:

- ig:ProductLevelAD
  - Handles the commercial intent and breaks it down into CFS requirements.
- ig:ServiceLevelAD
  - Translates CFS intent into Service Intent before forwarding it to ig:ResourceLevelADs.
- ig:ResourceLevelAD
  - Implements RFS by orchestrating network, computing, and application resources.

Autonomous Domain Workflow

- ig:ProductLevelAD
  - Defines business intent for the 5G slice.
  - Maps to CFS (e.g., "Ultra-Reliable Low-Latency Service").
- ig:ServiceLevelAD
  - Maps CFS into technical service requirements.
  - Assigns QoS policies (e.g., prioritization, traffic engineering).
- Ig:ResourceLevelAD
  - Allocates network RFS for the service.
  - Implements edge computing deployment for latency reduction.

The intent processing model includes:

- ig:ProductLevelIntent Definition
  - I_P be the product intent at the Product Level AD.
  - CFS_k be the CFS fulfilling a given intent.
  - $U_P(I_P)$ be the ont:UtilityFunction measuring the intent fulfillment.
- Each ig:ProductLevelAD defines a high-level intent:

$$I_P = \{(T_k, P_k)\}$$

where:

- T_k is the Target Delivery Expectation for CFS_k.
- P_k is the Property Expectation that must be met.
- The CFS mapping function transforms product intent into CFS:

$$CFS_k = f_P(I_P)$$

$$U_P = w_1F_1(CFS) + w_2F_2(CFS) + \cdots. + w_nF_n(CFS)$$

where w_i are weights for different product objectives (e.g., revenue, KPI compliance).

© TM Forum 2025. All Rights Reserved.

ig:ServiceLevelIntent Definition: At the ig:ServiceLevelAD, business intent is refined into technical service intent:

$$I_S = g_P(CFS)$$

The function g_P represents the intent transformation process from the CFS level to the ig:ServiceIntent **(I_S)** decomposition. g_P is a function that translates business intent from a product perspective into technical service **intent** decomposition. It maps CFS-level requirements (which are business-oriented) into service-level specifications (which are more technical and operational). The subscript P indicates that this function operates from the product level. The function g_P translates this into a ig:ServiceIntent I_S like: Low Latency ≤ 5ms, guaranteed Bandwidth ≥ 100 Mbps, and QoS Priority Level = High.

Suppose a CFS is "Premium 5G Connectivity", which is a commercial offering.

$$I_S = \{(T_m, P_m)\}$$

where:

- T_m = Target service-level expectation.
- P_m = Service quality requirement (e.g., latency, availability).

Intent Properties

The 'ig:ig:' property defines a hierarchical relationship between intents, meaning that a higher-level intent can be decomposed into multiple sub-intents. The ig:hasSubIntent provide the following:

- Intent Decomposition:
  - In an intent-driven model, a high-level business intent (e.g., "Provide 5G connectivity with ultra-low latency") must be broken down into smaller, more manageable intents across different domains.
  - Example:
    - "Optimize Network Slicing."
    - "Allocate Dedicated Bandwidth."
    - "Ensure Edge Computing Resource Availability."
    - High-Level Intent: "Ensure 5G URLLC."
    - Sub-Intents
- Intent Processing at Multiple Levels:
  - A ig:ProductIntent may generate multiple ig:ServiceIntents.

© TM Forum 2025. All Rights Reserved.

- A ig:ServiceIntent may generate multiple ig:ResourceIntents.
- Each sub-intent is processed within its respective AD.

The ig:hasSubIntent work in the model as follows:

1. Ig:ProductIntent (Handled in ig:ProductLevelAD)
   - Ig:hasSubIntent breaks it into multiple CFSs.

ig:ServiceIntent (Handled in ig:ServiceLevelAD)

- ig:hasSubIntent breaks it into multiple RFS.

3. ig:ResourceIntent (Handled in ig:ResourceLevelAD)

- ig:hasSubIntent ensures the intent is mapped to resource allocations.

Service ont:UtilityFunction

The Service Utility Function evaluates how well the ig:ServiceIntent is met:

$$U_S = w_1F_1(I_s) + w_2F_2(I_s) + \cdots .. + w_nF_n(I_s)$$

where $F_i(I_s)$ measures KPI adherence, service orchestration efficiency, and cost.

- ig:ResourceIntent Definition

At the ig:ResourceLevelAD, service intent decomposition is mapped to RFSs:

$$RFS_q = h_S(I_s)$$

This function h_S represents the mapping of ig:ServiceIntent decomposition into igResourceIntent. h_S translates service intent into RFSs, which are directly bound to the underlying infrastructure (network functional elements). The subscript *S* indicates that this function operates from the service layer to the resource layer. Suppose a ig:ServiceIntent is "Low-Latency 5G Data Routing". The function h_S translates this into resource-specific RFS,

$$I_R = \{(T_q, P_q)\}$$

where:

- T_q = Resource target expectation (e.g., allocated bandwidth, compute capacity).
- P_q= Technical constraints (e.g., network routing policies).

© TM Forum 2025. All Rights Reserved.

The Resource ont:UtilityFunction ensures resource efficiency and network optimization:

$$U_R = w_1F_1(RFS) + w_2F_2(RFS) + \cdots . + w_nF_n(RFS)$$

where $F_i(RFS)$ considers network load, energy efficiency, and service orchestration feasibility as shown in Table 38.

Table 38: CFS and RFS Function Transformation

Function	Transformation	Meaning
g_P	$I_S = g_P(CFS)$	Transforms CFS (business-level service) into ig:ServiceIntent
h_S	$RFS_q = h_S(I_S)$	Transforms ig:ServiceIntent into RFS

Autonomous Domain Closed-Loop Control Mechanism

The closed loop control is represented as a sequence of functional mappings intra and inter layers:

- $BusinessToServiceFlow$: ig: $BusinessIntent \rightarrow$ ig: $ServiceIntent$
- $ServiceToResouceFlow$: ig: $ServiceIntent \rightarrow$ ig: $ResouceIntent$
- $ResouceToBusinessFlow$: ig: $ResourceIntent \rightarrow$ ig: $BusinessIntentReport$

Closed loop control consist of intent to communicate and distribute requirements and intent reports to close the loop with feedback and allows negotiation of requirements, thus collaborative refinement and optimization of intent.

The intra closed loops are the AD closed loops, and these domains can be either Business, Service and Resource layer, these are the loops implemented and coordinated by IMFs. Each AD implements a Closed-Loop Control System with four key phases:

- Awareness Phase
  - Each AD continuously monitors its environment to detect changes in network conditions, SLA performance, and customer demand:

    $$S_t = \{CFS_t, RFS_t, Network_t\ , Computing_t\}$$

    where S_t represents the real-time state of the system at time t.

- Analysis Phase
  - *ig:AIAgent* analyze the gap between actual performance and intent expectations:

    $$\Delta I = I_{Expected} - I_{Observed}$$

  - where:

© TM Forum 2025. All Rights Reserved.

- $I_{Expected}$ = Original product intent targets.
- $I_{Observed}$ = Measured system performance.

- Decision-Making Phase
  - AD selects the optimal corrective action:

$$Act_t = \underset{Act}{argmax}\, U(Act|S_t)$$

where:

- Act_t is the selected corrective action.
- $U(Act|S_t)$ measures how well the action improves intent fulfillment.

- **Execution Phase**
  - The selected action is applied to adjust the network or service configuration.

Cross-Domain Coordination and Negotiation Model

Agents interact within a domain to coordinate local domain intent, collaborate within the same domain to perform distributed tasks the so-called horizontal Interactions, or intra domain interaction. Agent also interacts across domains to exchange information to fulfill the global intent, exchange information across domains to support end-to-end service delivery, the so-called vertical domain interactions, or inter domain interaction.

Horizontal Interactions

- Within a single AD, multiple ig:AIAgent collaborate to manage CFS or RFS execution.
- For example, multiple ig:ResourceLevelADs coordinate spectrum allocation and network routing to optimize service delivery.

Vertical Interactions

- Ig:ProductLevelAD → ig:ServiceLevelAD → ig:Resource-LevelAD
- Each AD refines the intent before passing it down to the next level.

Intent Negotiation Game

Each AD participates in a cooperative game where it negotiates with other ADs to optimize shared objectives:

$$max \sum_{l} U_l$$

subject to:

- Cross-domain SLA constraints.
- Resource availability limits

© TM Forum 2025. All Rights Reserved.

A provider provisions a network slice for an enterprise customer with:

- Guaranteed 5ms latency.
- Minimum 100 Mbps bandwidth.
- Closed-Loop Monitoring
    - ig:AIAgent continuously monitor latency.
    - If latency exceeds 5ms, AI triggers resource reallocation.

Intent Management Function (IMF) Model

The **IMF** model components include:

- Coordinates *ig:AIAgent* within a single AD (intra-AD coordination).
- Manages interactions between different ADs (inter-AD coordination).
- Handles intent propagation, negotiation, and optimization across hierarchical layers.

Entities & Definitions

Let

- AD_i - Autonomous Domain (e.g., Access, Transport, Core, etc.,).
- IMF_i - IMF within domain AD_i.
- $Ag_{i,j}$ - is the jth agent that support a specific function in AD_i.
- I_i - is the **intent** that is managed by IMF_i within AD_i.
- CFS - CFS is managed by higher-level ADs.
- RFS - RFS is managed at the resource level.

Each IMF:

- Coordinates multiple ig:AIAgents within its AD (intra-AD coordination).
- Negotiates with IMFs of other ADs (inter-AD coordination).
- Ensures closed-loop decision-making for self-optimization and autonomous control.

Formalizing Intent Processing in IMF

Each *IMF* interprets, refines, and executes intent:

$$I_i = \{(T_k, P_k)\}$$

where:

- T_k = Target expectation (e.g., bandwidth allocation, latency).
- P_k = Property constraint (e.g., QoS requirement, SLA).

© TM Forum 2025. All Rights Reserved.

The IMF decomposes intent into service or resource-level requirements respectively:

$$S_i = f(I_i)$$

where S_i represents CFS or RFS.

Each IMF applies an optimization function to determine the best actions:

$$Act_i = \underset{Act}{argmax}\, U(Act|S_i)$$

The above formula is a mathematical representation of decision-making optimization used in autonomous network operations with AI-driven agents. It describes how an ig:AIAgent selects the optimal action Act_i to maximize the ont:UtilityFunction $U(A|S_i)$ given the current system state S_i.

where:

- Act_i → The action chosen by the AI agent at time i by the IMF
- $\underset{Act}{argmax}$ → find the argument (action) Act that maximizes the function.
- Act → The set of all possible actions the AI agent can take.
- $U(Act|S_i)$→ The ont:UtilityFunction, which measures the expected benefit or performance of acting, Act when the system is in state S_i.
- S_i → The current state of the network, system, or resource domain

Interpretation in AI and Autonomous Networks. This equation is a RL-based decision rule where an AI agent:

- Observes the network state S_i which could include bandwidth usage, latency, failures, or service quality.
- Evaluates possible actions Act —such as rerouting traffic, allocating additional bandwidth, adjusting QoS settings, etc.
- Selects the action that maximizes the expected utility $U(Act|S_i)$ ensuring that intent-based service requirements are met while optimizing network performance.

Example in Intent-Driven Networks Optimization

AI Agent manages bandwidth allocation. Given a state S_i (e.g., high traffic congestion on a link), the agent considers multiple possible actions:

- Act_1 : Increase bandwidth on the congested link
- Act_2: Reroute traffic through an alternative path
- Act_3: Reduce non-priority traffic through policy adjustments

Each action has an associated utility value based on its expected impact on latency, packet loss, and overall service quality. The ig:AIAgent then chooses the

© TM Forum 2025. All Rights Reserved.

action Act_i that maximizes utility, ensuring that the network operates efficiently while adhering to business and ig:ServiceIntents.

MAS for Intra-AD Coordination

In a MAS, multiple ig:AIAgents can negotiate and collaborate using this decision-making principle. Each agent independently applies the formula but also interacts with other agents to align their intent-based objectives. For example, an ig:IntentNegotiationAgent may select an action that not only maximizes its local utility but also contributes to a globally optimal network state, facilitating inter-domain coordination.

Each *IMF* manages multiple ig:AIAgents within a single AD_i:

$$Ag_{i,j} \in AD_i, \qquad j = 1,2, \dots \dots, n$$

where $Ag_{i,j}$ is an AI agent responsible for executing a specific function.

Each AI agent follows an MDP model:

$$M_{i,j} = (S, Act, T, R)$$

where:

- *S* = State of the network resources and service performance.
- *Act* = Action set for adjusting resources (e.g., bandwidth allocation, routing).
- *T(S, Act, S')* = State transition probability after action, *Act*.
- *R(S, Act)* = Reward function measuring intent fulfillment.

The optimal policy $\pi_{i,j}$ for each agent is:

$$\pi^*_{i,j} = arg \max_{\pi} \sum_{t=0}^{T} \gamma^t R(S_t, A_t)$$

where γ is the discount factor for balancing short-term vs. long-term gains.

Each IMF coordinates *ig:AIAgents* via Multi-Agent RL (MARL)**:**

$$\max_{\{A_{i,j}\}} \sum_{j=0}^{n} U_{i,j}$$

subject to:

- Resource constraints within AD_i.

© TM Forum 2025. All Rights Reserved.

Inter-AD Coordination (IMF-to-IMF Negotiation Model)

Each *IMF* interacts with other *IMF*s for cross-domain intent coordination.

$$IMF_i \leftrightarrow IMF_j \quad i \neq j$$

where IMF_i and IMF_j belong to different Autonomous Domains.

Intent Propagation Across ADs

Each *IMF* negotiates with others using a game-theoretic approach:

$$max \sum_{i=1}^{N} U_i$$

subject to:

- Cross-domain constraints (e.g., transport domain availability).
- Network-wide SLA compliance.

IMFs follow a multi-agent negotiation strategy:

- Propose intent adjustments if resource allocation is suboptimal.
- Negotiate service trade-offs (e.g., prioritizing ultra-low latency over bandwidth).
- Reach equilibrium where no IMF can improve without impacting another.

Example: Access, Transport, and Core Network Resource Coordination

The customer request is received at the product layer *IMF*, which is decomposed to *CFS*s and *RFS* (aka Network Service), then it is passed to the Service layer *IMF*. These are different ADs and the relevant *IMF* in each domain at the boundary of these domains perform the integration between these two ADs. Intent based *IMF* at the service layer enable the end-to-end integration programmatically through open APIs and data modelling tools for the access AD, Transport AD and Core AD as follows:

- Access AD, IMF_A detects congestion → requests additional bandwidth.
- Transport AD IMF_T negotiates bandwidth allocation from Core AD IMF_C.
- Core AD IMF_C adjusts network slicing to accommodate demand.
- IMFs finalize agreement and execute new network configuration**.**
- *managesIntent*: Links IntentManagementFunction to **Intent,** ensuring that each IMF is responsible for interpreting and handling intent**.**

- engagesInNegotiation: Links IntentManagementFunction to Negotiation, reinforcing IMF-based intent coordination across domains.

© TM Forum 2025. All Rights Reserved.

- IMF is also linked to Autonomous Domains **(***hasIMF***).**
- Agents (ig:AIAgent**)** remain responsible for optimizing Intent (ig**:**optimizesIntent)
- Intent properly flows across Product, Service, and Resource levels (ig:hasProductIntent**,** ig:hasServiceIntent**,** ig:hasResourceIntent**).**

IMFs should ensure:

- Autonomous resolution of resource conflicts.
- Optimal allocation of bandwidth across ADs.
- Dynamic adaptation based on real-time conditions.

Utility-Based Closed-Loop Control

Each *IMF* continuously monitors, analyzes, decides, and executes (MADE loop)**:**

Awareness (State Monitoring)

$$S_t = \{CFS_t, RFS_t, Resource_t, Network_t\}$$

- Real-time observation of network performance and service demand**.**

Analysis (Deviation Calculation)

$$\Delta I = I_{Expected} - I_{Observed}$$

- Measures the gap between expected and actual intent fulfillment**.**

Decision-Making (Action Optimization)

$$A_t = arg\max_{Act} U(Act|S_t)$$

- *IMF* selects the best action based on network constraints and SLA requirements.

Execution (Autonomous Policy Update)

- Applies AI-driven changes to the network, services, or resources.

Each *IMF* follows a reward function:

$$U = w_1F_1 + w_2F_2 + \cdots. + w_nF_n$$

where:

- F_1 = SLA compliance.
- F_2 = Latency reduction.
- F_3 = Cost efficiency.
- F_n = Other performance metrics.

© TM Forum 2025. All Rights Reserved.

The goal is to maximize global network efficiency:

$$\sum_{i=1}^{N} U_i \rightarrow max$$

AI Agent Intent-Based Reports

Let's consider an AI-driven network slicing use case in a 5G autonomous network:

Scenario: A ig:ServiceIntent specifies:

- Guaranteed 100 Mbps bandwidth for a specific service.
- Maximum latency of 10 ms.
- Redundancy requirements for fault tolerance.

AI Agent Intent-Based Report Generation

- Nodes - ig:AIAgent, network devices, service policies.
- Edges - Relationships between service intents, SLA compliance, and infrastructure.
- Attributes - Metrics like current bandwidth, latency, and fault status.

A partial graph snapshot (intent report) at time *T* might show:

- The network slice currently operates at 90 Mbps (below the required threshold).
- Latency has increased to 12 ms, indicating potential degradation.
- The ig:IntentMonitoringAgent detects the issue and triggers an update.

The ig:IntentNegotiationAgent might then reconfigure resources or request changes from the Resource-Level ig:AIAgent.

KG is used for AI Intent Reporting to enable

- Structured Representation - Provides a semantic model that integrates AI decision-making.
- Real-Time Monitoring - Captures the current state dynamically.
- Automated Inference - Enables AI reasoning to predict and proactively adjust network configurations.
- Inter-Agent Collaboration - Facilitates seamless coordination between ig:AIAgent in MAS.

AI Agent Report Function

- AI Agent MAS Intent-Based Report
    - ig:IntentReport in an AI-driven Intent Management System are not just static logs but evolving knowledge objects—integrated within a KG that continuously refines network operations through AI reasoning, multi-agent coordination, and real-time adaptation.
    - An ig:AIAgent intent-based report as a KG represents the current state of an intent-driven AN in a structured format.

© TM Forum 2025. All Rights Reserved.

- The partial graph focuses on only the relevant aspects of the system at a given point in time, ensuring real-time compliance tracking, policy adaptation, and network self-optimization.
- This enables AI-driven intent-based service management to continuously monitor, adapt, and optimize network operations autonomously.
- An ig:AIAgent intent-based report is structured as a (or a partial graph) because it represents the intent management state in a way that captures relationships, dependencies, and evolving network conditions at a specific point in time. The following subsection provides explanation of how this concept works.

Generate Report Function:

ig:IntentMonitoringAgent instances track compliance and create ig:IntentReport nodes that reference met:RuntimeMetric observations

The function generates an intent report

The function $GenerateReport(i, r)$ generates an intent report

$r \in IntentReport$ for intent $i \in Intent$

$$GenerateReport(i, r) = \{ReportData \mid SatisfiesRequirement(i)\}$$

i - refers to a specific intent

r - refers to a report that is generated for a specific intent *i*

© TM Forum 2025. All Rights Reserved.

12. Appendix B: Simulation vs. Emulation vs. Digital Twin – Tradeoffs in Resource-Level Coordination

Simulation and emulation are often referenced interchangeably in AI-assisted network automation, but their technical scope, realism, and applicability differ significantly. This appendix compares these methods along with the concept of digital twins, clarifying their roles and tradeoffs in the resource layer of intent-based autonomous networks (ANs) [21],[34].

Conceptual Relationships – Figure 8 is a Venn diagram that illustrates the conceptual and functional overlap between simulation, emulation, and digital twins in resource-level orchestration:

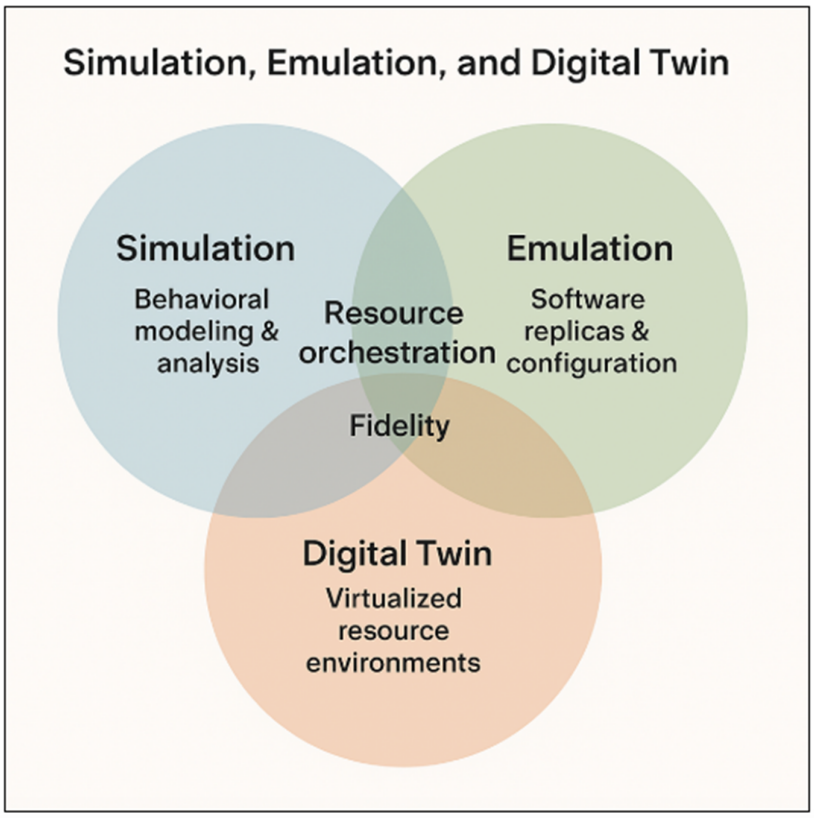

Figure 8: Three concepts intersect and complement one another in resource-level orchestration.

© TM Forum 2025. All Rights Reserved.

Simulation is useful for testing intent and policy constraints using abstracted models. Emulation replicates real network behavior, providing protocol accuracy but lacks real-time telemetry. Digital Twins offer a composite view by leveraging network topology, configuration, and service mappings to inform both simulation and emulation.

Key Interpretations:

- Simulation ∩ Digital Twin: Enables policy verification over modeled topologies (e.g., TMF686).
- Emulation ∩ Digital Twin: Enables protocol testing within a mirrored network model.
- Simulation ∩ Emulation: Combines model planning with real-behavior execution.
- All Three: Represents the ideal state of closed-loop assurance and dynamic intent evaluation.

12.1. Hybrid Telemetry / Partial-Twin Flow

Figure **9**Figure 9 depicts hybrid telemetry and partial twin flow as outlined below and summarize in Table 39:

1. Southbound telemetry (teal) flows upward from each network element to an Operational Data Broker.
2. The broker normalizes vendor YANG into the Dynamic Service & Inventory Map (orange).
3. The Ontology / SKOS Layer (blue) abstracts cross-vendor concepts.
4. Policy & Simulation Engines (violet) analyze the semantic graph, simulate what-if scenarios, and send northbound feedback (grey dotted arrows) as policy or intent adjustments back toward the resource-view.

© TM Forum 2025. All Rights Reserved.

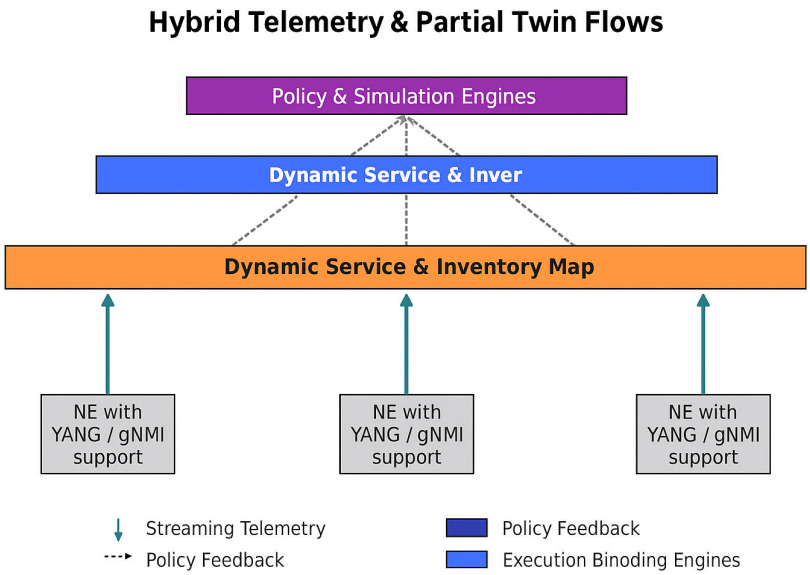

Figure 9: Hybrid Telemetry and Partial Twin Flow

Materialization Performance

While the partial twin model supports ingesting normalized telemetry (e.g., from YANG Push or gNMI), the architecture does not require full materialization at ingestion. Instead, materialization of knowledge graph triples can be deferred or selectively applied:

- Telemetry-first ingestion into time-series or NoSQL stores
- On-demand graph projection using virtual views or cached partial graphs
- SHACL-based validation for intent-relevant shapes, avoiding full ontology reasoning at load time
  This hybrid strategy balances expressiveness and performance, especially when integrating ig:AIAgent with high-frequency data sources across resource domains.

Table 39: Diagram Layer Outline Description

Diagram Layer / Arrow	Description in Diagram	Correspondence in Write-up
Teal Arrows (↑)	Streaming telemetry from devices	"Southbound telemetry (teal) flows upward..."
Orange Layer	Dynamic Service & Inventory Map	"Broker normalizes vendor YANG into..."
Blue Layer	Ontology / SKOS abstraction layer	"Ontology abstracts cross-vendor concepts..."
Violet Box	Policy & Simulation Engines	"Engines simulate what-if scenarios..."
Grey Dotted Arrows (↓)	Policy feedback / adjustments	"Send northbound feedback... as policy or intent adjustments..."

© TM Forum 2025. All Rights Reserved.

12.1.1. Ontology Constructs

Ontology constructs the Intent Translator Module as outlined below and summarized in Table 40:

1. **Intent Expression Ontology Class**

- Captures business/technical goal, constraint, and KPI semantics
- Formalizes IntentStatement, IntentGoal, IntentConstraint, and IntentTarget

2. **Intent Context & Domain Binding**

- Uses hasScope, appliesTo, and hasDomain properties to define:
  - Where the intent applies (Product, Service, or Resource AD)
  - The scope for resolution (e.g., slice, function, NF set)

3. **Intent Transformation Rules**

- Defined using IntentToPolicyMapping or IntentActionPattern
- Includes transformation logic modeled as part of **semantic rule base** (e.g., via SHACL, SWRL, RIF)

Table 40: AI Agent Module Architecture

Subcomponent	Function
Intent Parser	Extracts declarative fields from natural/structured input (goal, metric)
Ontology Binder	Resolves parsed tokens to ontology classes (IntentGoal, IntentDomain, etc.)
Semantic Mapper	Translates intent graph patterns into candidate policies or orchestration patterns
Policy Generator	Forms actionable rule statements using predefined templates (e.g., PBC, TMF rules, YANG mapping)
Feedback Connector	Integrates with AI Monitoring Agent to learn translation efficacy and adapt mappings

12.2. Best Practices

Table 41 summarizes Simulation, emulation and digital twin comparison

- Use simulation for intent translation, resource planning, and validation against topology constraints.
- Use emulation to test device configurations or routing behavior in isolated or pre-production settings.
- Use domain-scoped digital twins for visual overlays, service-mapping verification, and proactive assurance.

© TM Forum 2025. All Rights Reserved.

Table 41: Simulation, Emulation and Digital Twin Comparison

Capability	Simulation	Emulation	Digital Twin (Current State)
Definition	Modeling hypothetical scenarios using abstracted representations	Replicating real network devices or software for realistic behavior	Logical overlay mirroring partial network state, topology, and behavior
Strengths	Lightweight, scalable; useful for intent/policy validation and what-if analysis	Accurate representation of protocol/device behavior	Topology visualization; service-impact validation; closed-loop assurance foundation
Limitations	Lacks vendor-specific protocol behavior; can't simulate full control plane logic	High setup cost; limited real-device software availability for access networks	Not real-time; may lack behavioral fidelity; depends on accurate inventory/topology data
Use Cases	Intent validation, path planning, policy conflict testing	Route convergence testing, configuration migration, protocol compliance checks	Topology overlay validation, service-to-resource impact analysis, intent realization checks
Integration Potential	Easily integrates with orchestration and policy APIs (e.g., TMF686)	Best in labs or testbeds; not ideal for dynamic live-data use	Composable with TMF APIs; supports ZTP, assurance overlays, and service simulation

© TM Forum 2025. All Rights Reserved.

13. Appendix C: Integration of AI Agent MAS Framework

The ig:IntentTranslatorAgent:

- Operates at both Product and Service Domains
- Interacts upstream with the ig:IntentExpressionAgent
- Outputs machine-readable policies to ig:IntentHandlerAgent
- Using the ontology as the semantic grounding model
- May use pre-trained LLMs or symbolic reasoning engines (e.g., SPARQL, SWRL) to resolve ambiguity or refine intent as shown in Table 42

Table 42: Formal Representation Snippet (OWL/Turtle-Aligned)

```
:IntentTranslatorAgent a :AIAgent ;
    :hasFunction :TranslateIntent ;
    :usesOntology :IG1421IntentModel ;
    :bindsIntent [
        a :IntentStatement ;
        :hasGoal :EnsureAvailability ;
        :hasConstraint :AvailabilityGT99_99 ;
        :appliesTo :ServiceSliceX ;
    ] ;
    :outputs [
        a :Policy ;
        :appliesTo :ServiceSliceX ;
        :definesTargetState :HighAvailabilityConfigured ;
    ] .
```

Alignment with MI CONOPS 2035-2050

The MI CONOPS 2035–2050 roadmap [40] emphasizes intelligence modernization through AI augmentation, multi-agent autonomy, and cognitive orchestration, aligning strongly with the intent-based MAS architecture presented in this document.

Intent as Declarative Control: The concept of expressing mission goals through declarative intents mirrors the telecom shift to declarative service management. Agent-Based Reasoning: ig:AIAgent contexts should perform proactive reasoning, prediction, and adaptation using knowledge bases—driven by structured observations and mission/intent specifications.

Dynamic Autonomy and Human-AI Teaming: The telecom MAS facilitates real-time decision support and domain coordination without centralized orchestration. Cross-Domain Collaboration: Interoperability across autonomous domains (in telecom and operations service/resource stacks) is a shared architectural requirement.

© TM Forum 2025. All Rights Reserved.

Intent Lifecycle Automation TMF IG1253C [41]: The roadmaps propose end-to-end lifecycle control of missions; this aligns with IMF intent cycle management within AI-driven telecom infrastructure.
These parallels validate the design maturity of the AI agent MAS model and underscore its applicability across sectors seeking resilient, real-time, and intent-driven automation.

Materialization Performance

While the partial twin model supports ingesting normalized telemetry (e.g., from YANG Push or gNMI), the architecture does not require full materialization at ingestion. Instead, materialization of knowledge graph triples can be deferred or selectively applied:

- Telemetry-first ingestion into time-series or NoSQL stores
- On-demand graph projection using virtual views or cached partial graphs
- SHACL-based validation for intent-relevant shapes, avoiding full ontology reasoning at load time.

This hybrid strategy balances expressiveness and performance, especially when integrating ig:AIAgent with high-frequency data sources across resource domains.

© TM Forum 2025. All Rights Reserved.

14. Appendix D: Carrier-Influenced BGP Path Selection

Modern wide-area networks rely on BGP not only within an operator's infrastructure but between carrier networks. The BGP best-path algorithm (RFC 4271) evaluates a sequence of attributes—many of which (e.g., LOCAL_PREF, route communities, AS-path prepends) embody *commercial or policy constraints* imposed by upstream and downstream providers. Consequently, the chosen path is frequently the most policy-compliant path, not the shortest. Any meaningful simulation or partial digital-twin that aims to predict traffic shifts must therefore:

- Reproduce the standards-defined decision process **and**
- Ingest at least a proxy of external-carrier attributes or historical observations, because these external policies materially affect the routes visible inside one's own administrative domain.

This reinforces why today's simulations typically adopt a *hybrid* approach—combining standards logic, vendor YANG models, and observed BGP attributes—to approximate behavior, while recognizing that 100 % fidelity remains an open challenge.

© TM Forum 2025. All Rights Reserved.

15. Appendix E: T-Box and A-Box

In ontology and knowledge representation (especially in Description Logics, OWL, and semantic web contexts), **T-Box** and **A-Box** (see

Table **43**) are two foundational components used to structure knowledge:

T-Box (Terminological Box)

- Defines the vocabulary and schema of the domain.
- Describes concepts (classes) and relationships (properties) between those concepts.
- Think of it as the **ontology's "dictionary"** or **schema**.
- Example:
    - Router ⊑ NetworkDevice (Every router is a network device)
    - hasPort: NetworkDevice → Port

A-Box (Assertional Box)

- Contains assertions about individual instances of the concepts defined in the T-Box.
- Captures factual data (i.e., the current state of the world).
- Example:
    - Router1: Router (Router1 is an instance of the class Router)
    - Router1 hasPort Port5

Table 43: Summary of T-Box and A-Box

Aspect	T-Box	A-Box
Role	Schema / vocabulary	Instance data / facts
Describes	Classes, relationships	Individuals, class membership
Example	Switch ⊑ NetworkDevice	Switch42: Switch
Analogy	Table definition in a database	Rows/records in the database table

Cautionary Note on Ontology (TBox) and Instance Data (ABox) in Knowledge Graphs

While the terminology of TBox (schema-level knowledge) and ABox (instance-level data) is standard in ontology literature, it is often unfamiliar outside the semantic web community. Therefore, it's clearer to refer to them collectively as components of the knowledge graph, distinguishing between schema and instance data when necessary.

© TM Forum 2025. All Rights Reserved.

Importantly, dynamic updates to the schema (TBox)—such as introducing new classes, properties, or changing existing relationships—can lead to reclassification or invalidation of existing instance data (ABox). In such cases, a partial or full rewrite of affected triples may be required. This poses significant performance implications, especially for large graphs.

Graph database performance also depends on the inferencing strategy:

- Materialization at ingestion (used in many open-source triple stores) can be computationally expensive during updates, as all inferences must be recomputed.
- Query-time inferencing, on the other hand, defers reasoning to runtime, and may be better suited for environments where instance data updates (e.g., telemetry from networks) occur frequently.

In practice, learning how inferencing works during ingestion can deepen understanding of the ontology's expressive power, but for high-frequency data updates, a query-time inference model is more scalable and responsive.

© TM Forum 2025. All Rights Reserved.

16. Appendix F: Standards-Aligned Intent Execution Template

This template is designed to ensure semantic clarity, operational traceability and interoperability between system expressing and realizing intents.

Intent Model RFC 9315 and RFC9316

1. **Intent Declaration Block**

(From RFC 9315 & RFC 9316: semantic classification + declarative form). This block is Fully supported by TMF TR290A [37]. The Intent model is outlined in Table 44

Table 44: Intent Model

Field	Description
intent-id	Globally or domain-unique identifier for the intent
intent-class	High-level classification: e.g., Optimization, Protection, Assurance, Experience, ResourceUtilization
actor	Who originated the intent (e.g., Operator, Application, Consumer, Orchestrator)
objective	Declarative goal (e.g., “latency < 20ms”, “maximize throughput”, “reduce signaling congestion”)
priority	Priority level (e.g., high, medium, best-effort
temporal-context	Optional: duration or validity window (start-time, end-time, event-triggered)

- TMF TR290A [37] directly supports this block.
- It defines fields such as:
  - actor
  - intentType
  - objective
  - priority
  - scope
  - target
- TR290 [36] provides the conceptual definitions and relationships.

Fully supported by TMF TR290A [37]

2. Policy & Semantic Parsing Layer

Table 45 shows the intent field description *(RFC 9315 Section 6: Policy derivation and validation logic)*

© TM Forum 2025. All Rights Reserved.

Table 45: Intent Field Description

Field	Description
Target-objects	Network elements, services, or logical entities to which the intent applies
resolved policy	Derived policy rules (e.g., threshold rules, placement constraints, routing hints)
validation-shape	SHACL/JSON schema ensuring conformance (optional)
intent-type	RFC 9316 Taxonomy type: e.g., Declarative, Query, Trigger, Control, Goal
intent-subtype	More specific form: e.g., ThresholdIntent, PerformanceIntent, PlacementIntent

- TR290A [36] defines semantics and context but does **not** provide a formal mapping or inference logic.
- TMF lacks formal definitions for:
  - Policy resolution engines
  - Semantic parsing rules
  - Rule-based vs ML-based translation

Partially supported in TMF. RFC 9315 and ETSI TS 104 036 support parsing and policy realization models.

3. Execution Binding Block

The intent functional blocks are summarized in Table 46 *(ETSI TS 104 036: Execution within RAN or infra domain)*

Table 46: Intent Functional Block

Field	Description
execution-agent	The functional block responsible for acting on the intent (e.g., AI agent, SMO function, RIC xApp, SDN controller)
mechanism	Execution type: AI-inferred, Rule-based, Closed-loop, Predefined-scenario
validation-shape	SHACL/JSON schema ensuring conformance (optional)
actuation-layer	KPIs or telemetry used to verify success (e.g., SLA latency, throughput, energy profile)
fallback-policy	Optional: action if the intent cannot be fulfilled (e.g., retry, notify, degrade gracefully)

- TR290 [36] and TR290A [37] stop at intent declaration.
- TMF does not yet define how execution agents (e.g., AI agents, orchestrators, SMO functions) bind to intent).
- TMF Open APIs (like TMF641, TMF642) can support actuation, but they're not part of TR290 [36].

© TM Forum 2025. All Rights Reserved.

Not defined in TR290 [36]. Use IETF RFCs and ETSI 104 036 as authoritative references here.

4. Runtime Monitoring and Assurance Block

Table 47 shows the observability points *(Supports feedback and closed-loop adaptation)*

Table 47: Observability Points

Field	Description
observability-points	Points in the system where intent compliance is monitored
violation-handlers	AI/analytics triggers for remediation
feedback-loop-id	Loop controller tracking this intent's realization lifecycle
status	Current lifecycle state: active, violated, suspended, fulfilled, expired
update-policy	How frequently the intent realization state is re-evaluated (e.g., interval, event-driven)

- TR290B [37] support intent reporting.
- It supports fields such as:
  - status
  - metrics
  - compliance result
  - violation state
  - feedback loop ID

Supported by TMF TR290B.

5. Example: Performance Optimization Intent for RAN (see Table 48)

Table 48: Performance Optimization Intent for RAN

Performance Optimization Intent for RAN
{ "intent-id": "intent-ran-optimize-latency", "intent-class: "Optimization", "scop": RANFunction", "actor": "OperatorPolicyEngine", "objective": "latency < 10ms", "priority: "high", "intent-type": "Goal", "intent-subtype"; ThresholdInent",

© TM Forum 2025. All Rights Reserved.

```
“execution-agent”: “RIC-xApp”,
“mechanism”; AI-inferred”,
“actuation-layer: “RAN”,
“compliance-metric”: [“latency-RTT”, UE-packet-delay”]
“status”; “active”,
“observability-points: [“gNodeB”, “RIC-KPI-Agent”],
“fallback-policy”: “alert + degrade-QoS-profile”,
“feedback-loop-id”: loop-ran-001”
}
```

TMF Intent Model Artifacts and Alignment

TMF intent artifact are outlined in Table 49.

Table 49: TMF Intent Artifact

TMF Doc ID	Title	Version	Purpose / Role in Intent Lifecycle	Alignment
TR290 [36]	Intent Common Model	v3.0.0	Provides an overview and common definitions of intent structure and lifecycle.	Aligns with RFC 9315's semantic model foundation; defines shared ontology elements.
TR290A [37]	Intent Common Model – Intent Expression	v3.0.0	Defines the model elements used for declaring intent (e.g., goal, actor, target, priority).	Direct mapping to intentType, objective, and scope fields from RFC 9316.
TR290B [38]	Intent Common Model – Intent Reporting	v3.0.0	Describes how to express intent realization and compliance through reporting constructs.	Aligns with SHACL-based validation, A-Box compliance feedback, and observability loops.

© TM Forum 2025. All Rights Reserved.

17. Terms & Abbreviations Used within this Document

17.1. Terminology

Term	Definition	Source

17.2. Abbreviations & Acronyms

Abbreviation / Acronym	Abbreviation / Acronym Spelled Out	Definition	Source
IMF	Intent Management Function		IG1251
AN	Autonomous Network		IG1253
AD	Autonomous Domain	TMF	IG1251
MAS	Multi-Agent Systems	Wikipedia	https://en.wikipedia.org/wiki/Multi-agent_system#:~:text=A%20multi%2Dagent%20system%20(MAS,a%20monolithic%20system%20to%20solve.
MDP	Markov Decision Processes	Article	https://ceur-ws.org/Vol-1588/paper1.pdf
CFS	Customer Facing Service	TMF	TMF071
RFS	Resource Facing Service	TMF	TMF071
SLA	Service Level Agreement	TMF	TMF071 GB922
KG	Knowledge Graph	Wikipedia	https://en.wikipedia.org/wiki/Knowledge_graph
AI	Artificial Intelligence	TMF	TMF071 GB1190 GB1003

© TM Forum 2025. All Rights Reserved.

18. References

Reference	Description	Source	Brief Use Summary
1	Intent in Autonomous Networks	TMF IG1253	"Intent in Autonomous Networks", v1.3.0.
2	Autonomous Networks Reference Architecture	TMF IG1251	"Autonomous Networks Reference Architecture", v1.0.1
3	Evolving Service Management Toward Intent-Driven Autonomous Networks	Ericsson Technology Review	Evolving Service Management Toward Intent-Driven Autonomous Networks, #3 March 2024, Jörg Niemöller, Elisabeth Müller, Massimiliano Mag giari, Kamal Maghsoudlou
4	Product Catalog Management API	TMF620	"Product Catalog Management API"
5	Service Catalog Management API	TMF633	"Service Catalog Management API"
6	Product Offering Qualification Management API	TMF679	"Product Offering Qualification Management API"
7	AI Management API	TMF915	"AI Management API"
8	Intent Management API	TMF921	"Intent Management API"
9	LLMs4OL: Large Language Models for Ontology Learning	arXiv:2307.1664 8v2 [cs.AI] 2 Aug 2023	H. B. Giglou, et al., "LLMs4OL: Large Language Models for Ontology Learning", August 2023
10	ReACT: "Synergizing Reasoning and Acting in Language Models"	https://arxiv.org /pdf/2210.03629	Y. Shunyu et. al., "Synergizing Reasoning and Acting in Language Models", ICLR 2023
11	"Cognitive Architecture for Language Agents (CoALA)"	Publish in transaction of Machine Learning March 2024, https://arx iv.org/abs/2309. 02427	T. R. Sumers, et al., "Cognitive Architecture for Language Agents (CoALA)"
12	"Autonomous Cognitive Entity (ACE) Framework"	https://medium. com/@dave-shap/autonomo us-agents-are-here-	D. Shapiro, "Autonomous Cognitive Entity (ACE) Framework", Medium Sept 2023,

© TM Forum 2025. All Rights Reserved.

Reference	Description	Source	Brief Use Summary
		introducing-the-ace-framework-a180af15d57c	
13	"Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects"	https://arxiv.org/abs/2401.03428	C. Yuheng, et al., "Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects", January 2024,
14	"Multi-agent System: A Survey from a Machine Learning Perspective"	https://www.cs.cmu.edu/~mmv/papers/MASsurvey.pdf	"Multi-agent System: A Survey from a Machine Learning Perspective", Autonomous Robotics Vol 8 No 3, July 2000,
15	"An Evaluation Survey of Knowledge-Based Approaches in Telecommunication Applications"	https://doi.org/10.3390/telecom5010006, https://www.mdpi.com/2653744	G. P. Koudouridis, et al., "An Evaluation Survey of Knowledge-Based Approaches in Telecommunication Applications", Telecom 2024, 5, 98–121.
16	"Towards Large Reasoning Models"	https://arxiv.org/pdf/2501.09686	X. Fengli, et al., "Towards Large Reasoning Models", Jan 2025,
17	UiPath	https://www.uipath.com/ai/agentic-ai	UiPath
18	Intelligent Agent	https://en.wikipedia.org/wiki/Intelligent_agent	Wikipedia https://en.wikipedia.org/wiki/Intelligent_agent
19	"What are AI agents"	https://www.ibm.com/think/topics/ai-agents	"What are AI Agents", IBM July 2024.
20	Is it an Agent, or Just a Program?: A Taxonomy for Autonomous Agents	https://www.researchgate.net/publication/221457111_Is_it_an_Agent_or_Just_a_Program_A_Taxonomy_for_Autonomous_Agents	Is it an Agent, or Just a Program?: A Taxonomy for Autonomous Agents

© TM Forum 2025. All Rights Reserved.

Reference	Description	Source	Brief Use Summary
21	"An Introduction to Multi-Agent Systems"	Book, An Introduction to Multi Agent System", 2009 Wiley	M. Wooldridge, "An Introduction to Multi-Agent Systems"
22	"Artificial Intelligence: A Modern Approach"	Book, "Artificial Intelligence: A Modern Approach" (4th Edition). Pearson, 2021.	S. Russell, et al.,. "Artificial Intelligence: A Modern Approach"
23		"A Roadmap of Agent Research and Development. Autonomous Agents and Multi-Agent Systems", 1998	N. R. Jennings, et al., "A Roadmap of Agent Research and Development. Autonomous Agents and Multi-Agent Systems"
24	On autonomous domain orchestration and intent-policy binding.	ETSI ZSM 002 V1.1.1	On autonomous domain orchestration and intent-policy binding.
25	Service Models Explained	RFC 8969	Service Models Explained (why device-specific automation is hard).
26	Aether/SD-RAN – examples where programmable infrastructure is enabling early AI-based automation.	ONF	Aether/SD-RAN – examples where programmable infrastructure is enabling early AI-based automation.
27	IETF gNMI Telemetry Specification	https://github.com/openconfig/reference/blob/master/rpc/gnmi/gnmi-specification.md	IETF gNMI Telemetry Specification

© TM Forum 2025. All Rights Reserved.

Reference	Description	Source	Brief Use Summary
28	Requirements for Streaming Telemetry in Networks	RFC 8641	Requirements for Streaming Telemetry in Networks
29	"Differentiates between service-level intent and operational enforcement abstractions"	RFC 9315	"Differentiates between service-level intent and operational enforcement abstractions".
30	"Layered topology models that imply coordination boundaries within and across domains".	RFC 8345	"Layered topology models that imply coordination boundaries within and across domains".
31	"Topology Management API"	TMF686	"Topology Management API"
32	Combining Ontologies and Markov Logic Networks for Statistical Relational Mobile Network Analysis"	https://ceur-ws.org/Vol-1588/paper1.pdf	Kasper Apajalahti1, et al, "Combining Ontologies and Markov Logic Networks for Statistical Relational Mobile Network Analysis", Feb 2016,
33	Framework for Network Anomaly	RFC9232	Framework for Network Anomaly
34	Intent Classification and Taxonomy	RFC 9316	Intent Classification and Taxonomy
35	Intent-Based Policies in the RAN	ETSI TS 104 036 V12.0.0	Intent-Based Policies in the RAN
36	“Intent Common Model”	TMF TR290	“Intent Common Model”, v3.0.0
37	“Intent Common Model - Intent Expression”	TMF TR290A	“Intent Common Model - Intent Expression”, v.3.0.0
38	“Intent Common Model - Intent Reporting”	TMF TR290B	“Intent Common Model - Intent Reporting”, v.3.0.0
39	“Intent Common Model - Vocabulary Reference”	TMF TR290V	“Intent Common Model - Vocabulary Reference”, v3.0.0

© TM Forum 2025. All Rights Reserved.

Reference	Description	Source	Brief Use Summary
40	Future Military Intelligence CONOPS and S&T Investment Roadmap 2035-2050, Feb 2019.	https://nsiteam.com/social/wp-content/uploads/2019/04/Future-MI-CONOPS-and-ST-Roadmap-2035-2050_2-20-2019_FINAL.pdf	Haugland, Edward L. Future Military Intelligence CONOPS and S&T Investment Roadmap 2035-2050, Feb 2019.
41	"Intent Life Cycle Management and Interface"	TMF IG1253C	"Intent Life Cycle Management and Interface", v1.3.0

© TM Forum 2025. All Rights Reserved.

19. Administrative Appendix

19.1. Document History

19.1.1. Version History

Version Number	Date Modified	Modified by:	Description of changes
1.0.0	14 Aug 2025	Rephael Benhamo	Initial release

19.1.2. Release History

Release Status	Date Modified	Modified by:	Description of changes
Pre-production	19 May 2025	Rosie Wilson	Initial Publication
Pre-production	27-Oct-2025	Adrienne Walcott	Updated to Member Evaluated status

19.2. Acknowledgments

This document was prepared by the members of the TM Forum ODA Production team.

Team Member (@mention)	Company	Role*
Rephael Benhamo	Barnet Communications	Author and editor
Dave Milham	TM Forum	Co-editor/reviewer
Brad Peters	NBN Co	Contributor
Dymtro Gassanov	TM Forum	Reviewer
Peter Skoularikos	Telekinetics	Reviewer
Kevin McDonnell	Huawei	Additional Input
Mohit Goel	Telstra	Reviewer

**Select from: Project Chair, Project Co-Chair, Author, Editor, Key Contributor, Additional Input, Reviewer*

© TM Forum 2025. All Rights Reserved.